

形式语言与自动机

Formal Languages and Automata Theory

图灵机

计算机科学与技术学院
哈尔滨工业大学（深圳）

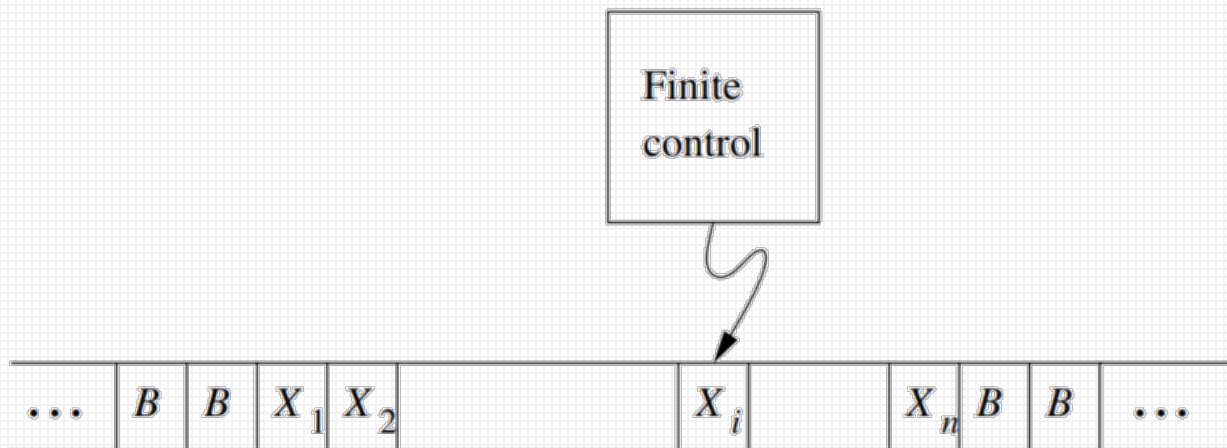


郑宜峰

- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- 图灵机的表示能力：语言、停机
- 扩展图灵机
- 受限图灵机

- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- 图灵机的表示能力：语言、停机
- 扩展图灵机
- 受限图灵机

- **图灵(Alan Mathison Turing)在1936年介绍了这样一个通用的计算模型，该模型具有以下两个性质：**
 - 该模型的每个过程都是有穷可描述的；
 - 过程必须是由离散的、可以机械执行的步骤组成。
- **图灵机是计算机的一种简单数字模型，尽管简单，但它具有模拟通用计算机的计算能力。**
 - 对算法和可计算性进行研究提供形式化描述工具。



- 图灵机有一个**有穷控制器**，一条**两端无穷的输入输出带**和一个**带头**(tape head)。
- 带划分为单元格，每个单元格可以放置一个符号。
 - 初始时，带上的符号由输入符号(input symbol)和空格符号B(blank symbol)组成
 - 空格符号是不同于输入符号的带符号(tape symbol)。除了空格符号外，也可以定义其他的带符号(tape symbol)
- **带头每次根据当前状态和带头处单元格的符号内容，根据转移规则选择下一个动作。**
 - 每个动作都包括下一个状态，修改带头处单元格的符号以及带头向左或向右移动一个单元格。
 - 带头最开始指向最左边的非空符号，即图中的 X_1

- 图灵机 (TM, Turing Machine) M 为七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

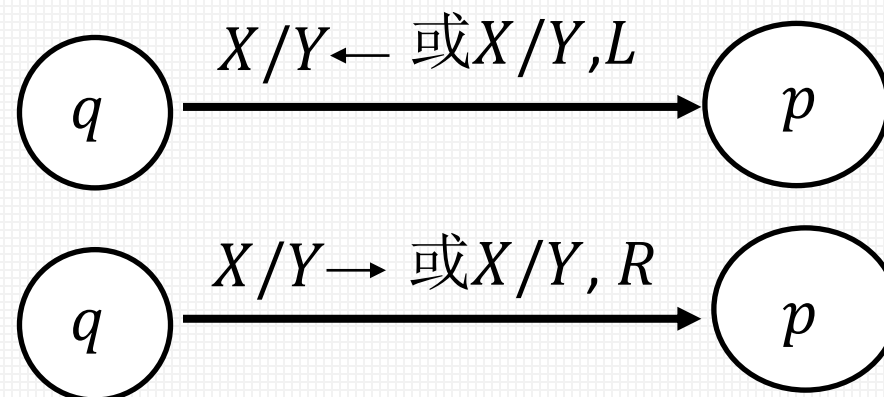
- (1) Q : 有穷状态集
- (2) Σ : 有穷输入符号集 (input alphabet)
- (3) Γ : 有穷带符号集 (tape alphabet), 且总有 $\Sigma \subseteq \Gamma$
- (4) $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$: 转移函数
- (5) $q_0 \in Q$: 初始状态
- (6) $B \in \Gamma - \Sigma$: 空格/空白符号 (Blank Symbol)
- (7) $F \subseteq Q$: 接受状态集

图灵机的动作及状态转移图

- 有穷控制器状态为 q 且带头读入符号 X 时，若有

$$\delta(q, X) = (p, Y, L/R)$$

则状态移到 p ，带头写符号 Y 并左或右移一格。图示为



- 和PDA类似，可以定义图灵机的瞬时描述ID，用于刻画图灵机的动作。
- 定义：用全部非空格符号、当前状态及带头位置，定义图灵机的瞬时描述(ID)：

$$X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n$$

- (1) q 是图灵机的当前状态；
 - (2) 带头在左起第 i 个非空格符 X_i 上；
 - (3) $X_1 X_2 \cdots X_n$ 是从最左到最右非空格内容
- ID转移符号 $\vdash_M$ 。

若某ID是从另一个经有限步转移而得到的，记为 $\vdash_M^*$ 。若 M 已知，简记为 $\vdash$ 和 $\vdash^*$ 。

- 左移动：如果 $\delta(q, X_i) = (p, Y, L)$ ，定义ID转移为

$$X_1 \cdots X_{i-1} \textcolor{blue}{q} X_i \cdots X_n \vdash_M X_1 \cdots X_{i-2} \textcolor{red}{p} X_{i-1} \textcolor{violet}{Y} X_{i+1} \cdots X_n$$

表示 M 做一次向左移动（消除 X_i 打印 Y ，并且左移，状态变成 p ）

- 左移动：如果 $\delta(q, X_i) = (p, Y, L)$ ，定义ID转移为

$$X_1 \cdots X_{i-1} \textcolor{blue}{q} X_i \cdots X_n \vdash_M X_1 \cdots X_{i-2} \textcolor{red}{p} X_{i-1} \textcolor{violet}{Y} X_{i+1} \cdots X_n$$

表示 M 做一次向左移动（消除 X_i 打印 Y ，并且左移，状态变成 p ）

- 特殊情况



为表明带头指向了空格符 B ，
包含空格符 B 在ID中。

– $i = 1, \textcolor{blue}{q} X_1 \cdots X_n \vdash_M \textcolor{red}{p} B \textcolor{violet}{Y} X_2 \cdots X_n$

– $i = n$ 且 $Y = B, X_1 \cdots X_{n-1} \textcolor{blue}{q} X_n \vdash_M X_1 \cdots X_{n-2} \textcolor{red}{p} X_{n-1}$

- 左移动：如果 $\delta(q, X_i) = (p, Y, L)$ ，定义ID转移为

$$X_1 \cdots X_{i-1} q X_i \cdots X_n \vdash_M X_1 \cdots X_{i-2} p X_{i-1} Y X_{i+1} \cdots X_n$$

表示 M 做一次向左移动（消除 X_i 打印 Y ，并且左移，状态变成 p ）

- 特殊情况

$$- i = 1, q X_1 \cdots X_n \vdash_M p B Y X_2 \cdots X_n$$

$$- i = n \text{ 且 } Y = B, X_1 \cdots X_{n-1} q X_n \vdash_M X_1 \cdots X_{n-2} p X_{n-1}$$

为表明带头指向了空格符 B ，
包含空格符 B 在ID中。

原先的最右符号 X_n 被替换为了空格符 B ，成为右边无穷空格符的一部分，因此从ID中去除。

- 右移动：如果 $\delta(q, X_i) = (p, Y, R)$, 定义ID转移为

$$X_1 \cdots X_{i-1} \textcolor{blue}{q} X_i \cdots X_n \vdash_M X_1 \cdots X_{i-1} \textcolor{violet}{Y} \textcolor{red}{p} X_{i+1} \cdots X_n$$

表示 M 做一次向右移动（消除 X_i 打印 Y ，并且右移，状态变成 p ）

- 右移动：如果 $\delta(q, X_i) = (p, Y, R)$ ，定义ID转移为

$$X_1 \cdots X_{i-1} q X_i \cdots X_n \vdash_M X_1 \cdots X_{i-1} Y p X_{i+1} \cdots X_n$$

表示 M 做一次向右移动（消除 X_i 打印 Y ，并且右移，状态变成 p ）

- 特殊情况

为表明带头指向了空格符 B ，
包含空格符 B 在ID中。

– $i = n$, $X_1 \cdots X_{n-1} q X_n \vdash_M X_1 X_2 \cdots X_{n-1} Y p B$

– $i = 1$ 且 $Y = B$, $q X_1 \cdots X_{n-1} X_n \vdash_M p X_2 \cdots X_n$

示例

- 右移动：如果 $\delta(q, X_i) = (p, Y, R)$ ，定义ID转移为

$$X_1 \cdots X_{i-1} q X_i \cdots X_n \vdash_M X_1 \cdots X_{i-1} Y p X_{i+1} \cdots X_n$$

表示 M 做一次向右移动（消除 X_i 打印 Y ，并且右移，状态变成 p ）

- 特殊情况

为表明带头指向了空格符 B ，
包含空格符 B 在ID中。

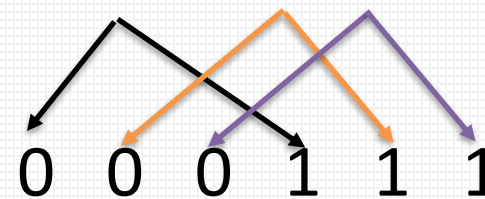
$$- i = n, X_1 \cdots X_{n-1} q X_n \vdash_M X_1 X_2 \cdots X_{n-1} Y p B$$

$$- i = 1 \text{ 且 } Y = B, q X_1 \cdots X_{n-1} X_n \vdash_M p X_2 \cdots X_n$$

原先的最左符号 X_1 被替换为了空格符 B ，成为左边无穷空格符的一部分，因此从ID中去除。

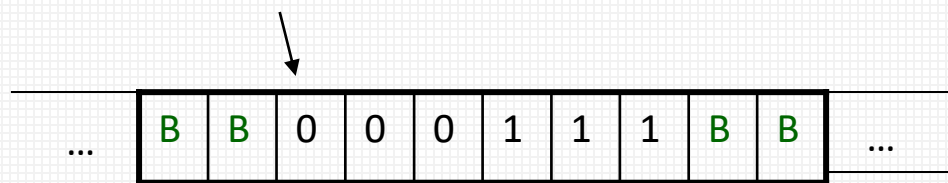
示例

- 例1:设计识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机。

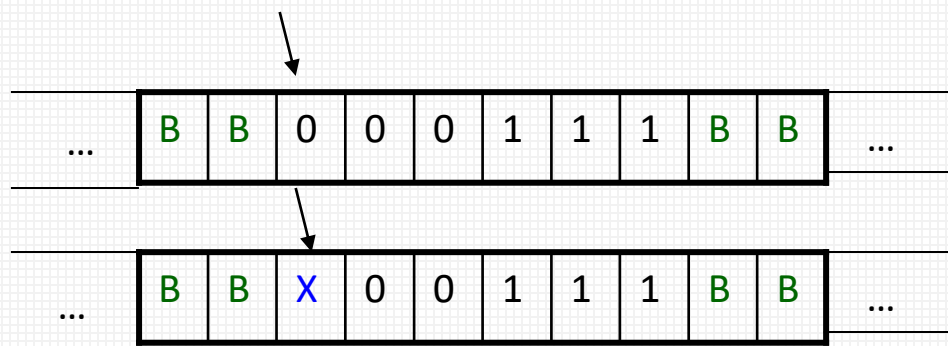


- 思路：
 - 把对应的、匹配的、成对的0和1进行标记(用X标记0，Y标记1)
 - 首先，将前面第一个0标记为X，然后往后面寻找第一个1，标记为Y。
 - 再回头向前去找第二个0，先要跳到第一个X，然后改变读头移动的方向，去寻找下一个0，再把它标记为X。接着跳过所有的0和Y,将遇到的第一个1标记为Y。
 - 重复这个循环，当所有的0和所有的1标记完后，若它们的数量相等且0在1前面，则进入接受状态。

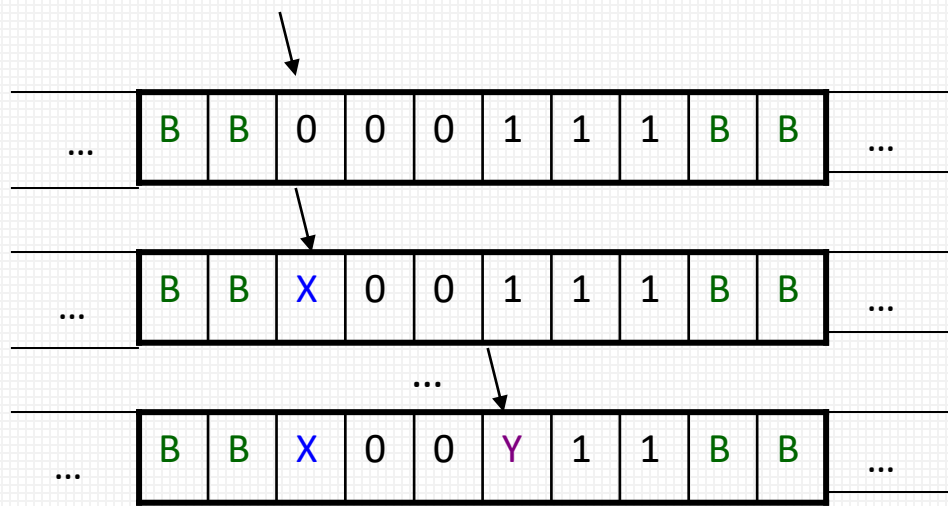
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



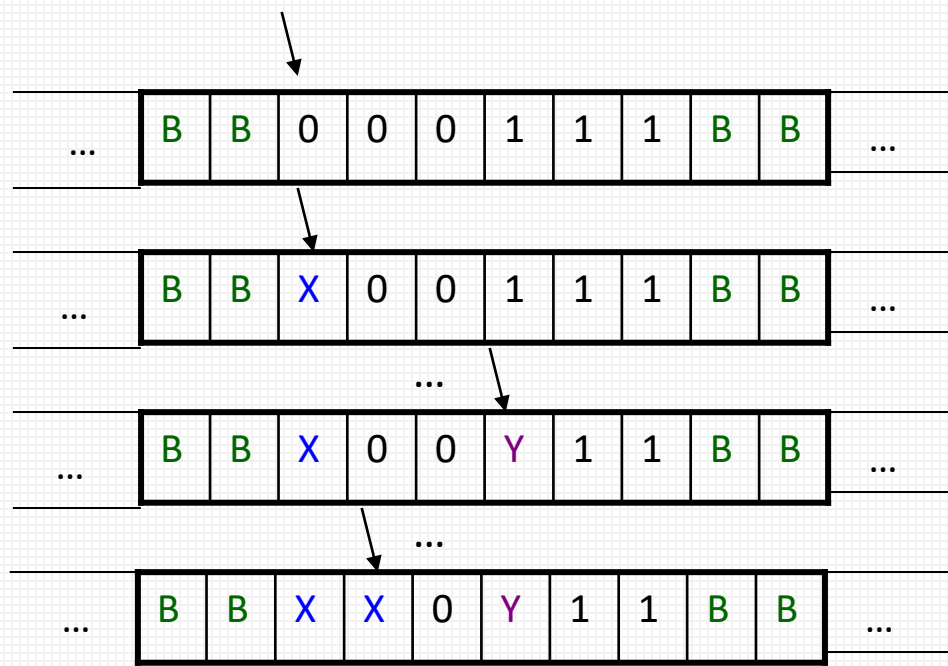
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



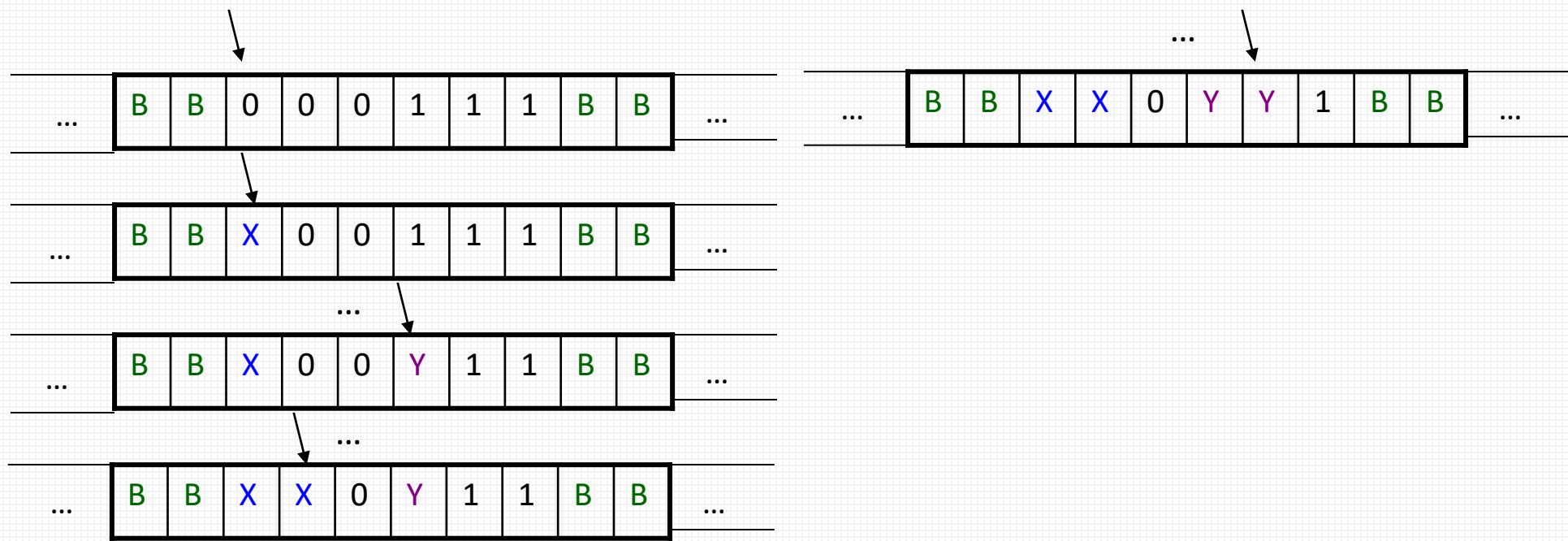
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



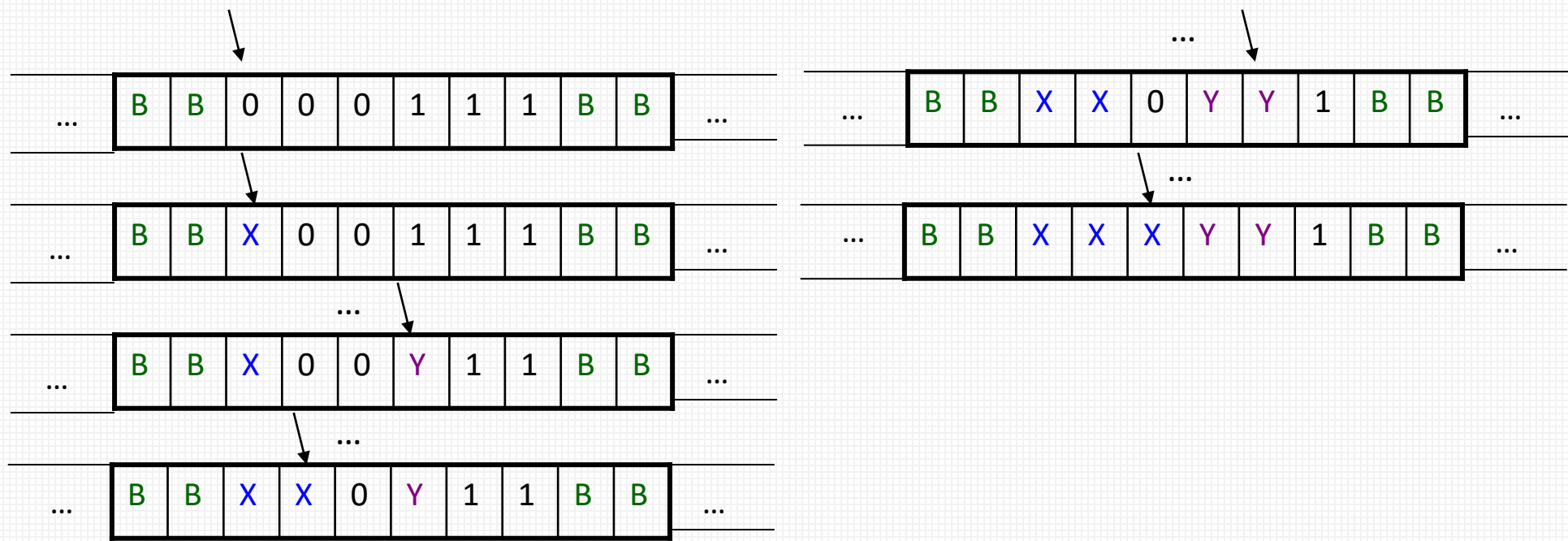
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



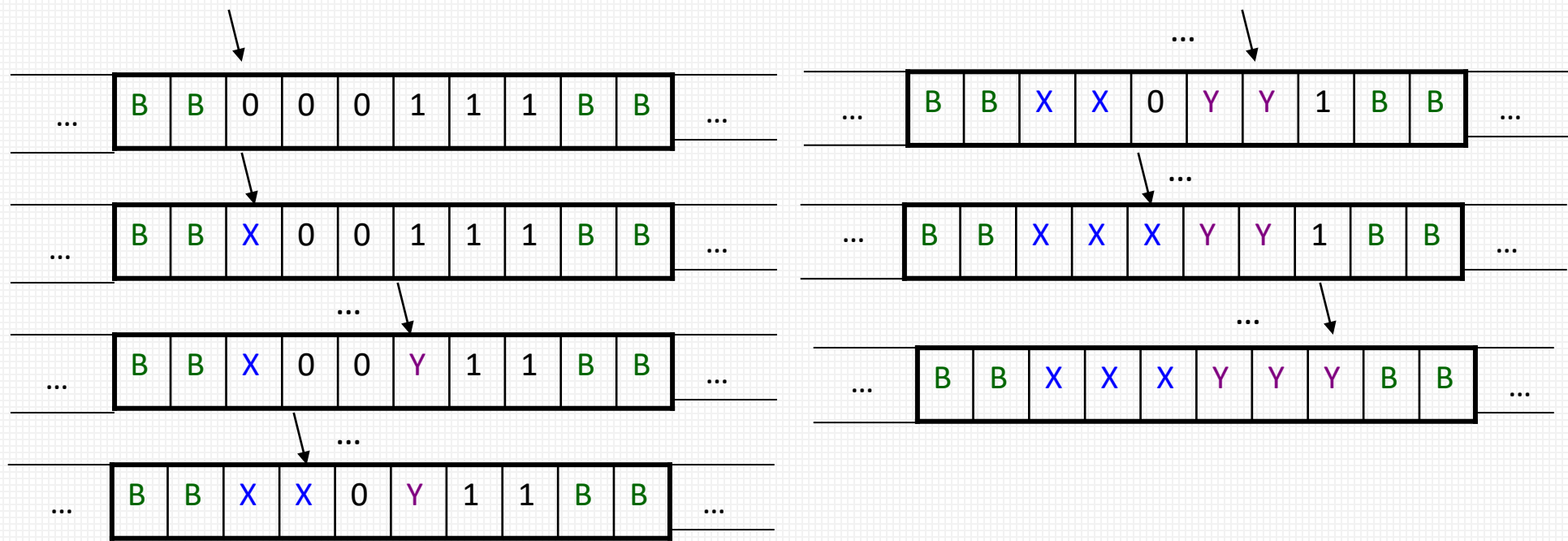
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



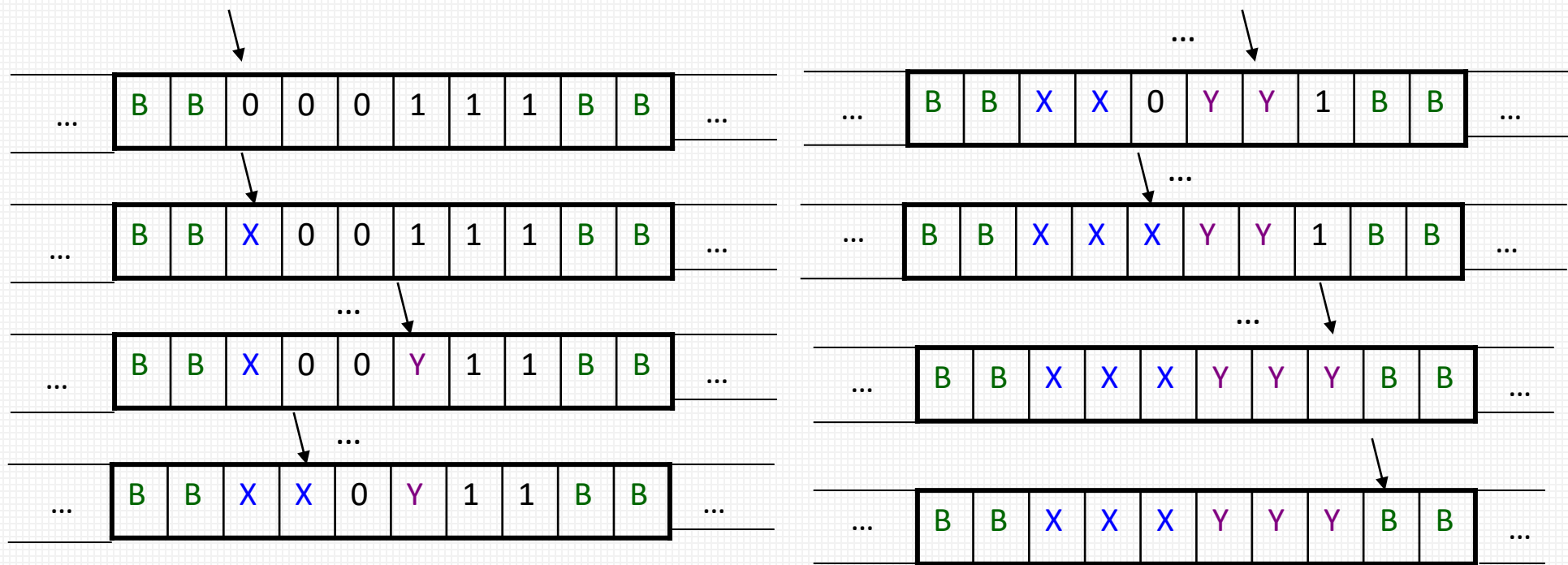
- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例

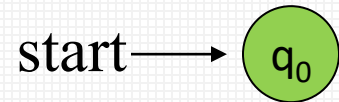


- 例1: 设计识别 $\{0^n 1^n \mid n \geq 1\}$ 的图灵机。
- 思路分析: 以 $w = 000111$ 为例



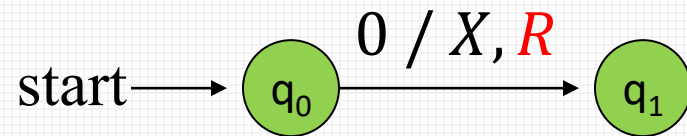
识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机

1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动

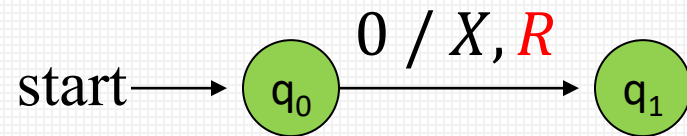


识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机

1. 处于 q_0 , 若待读入的为0, 则将0标记为 X , 向右移动

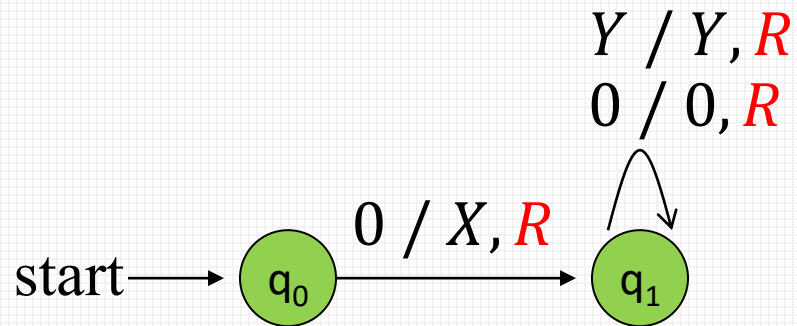


识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机



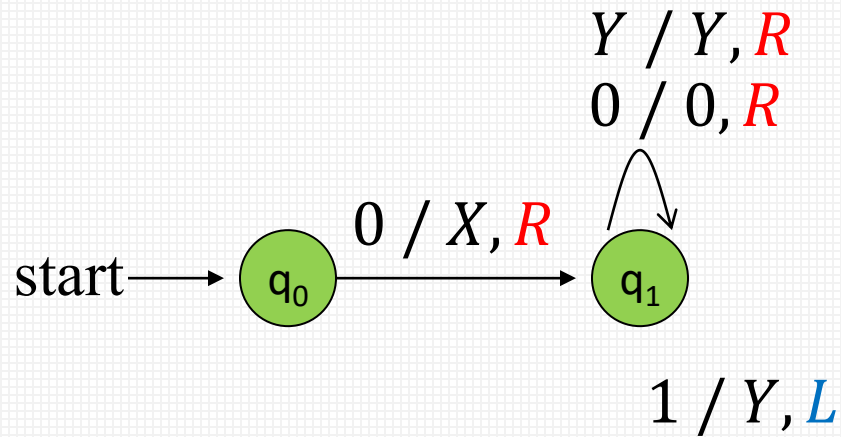
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动

识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机

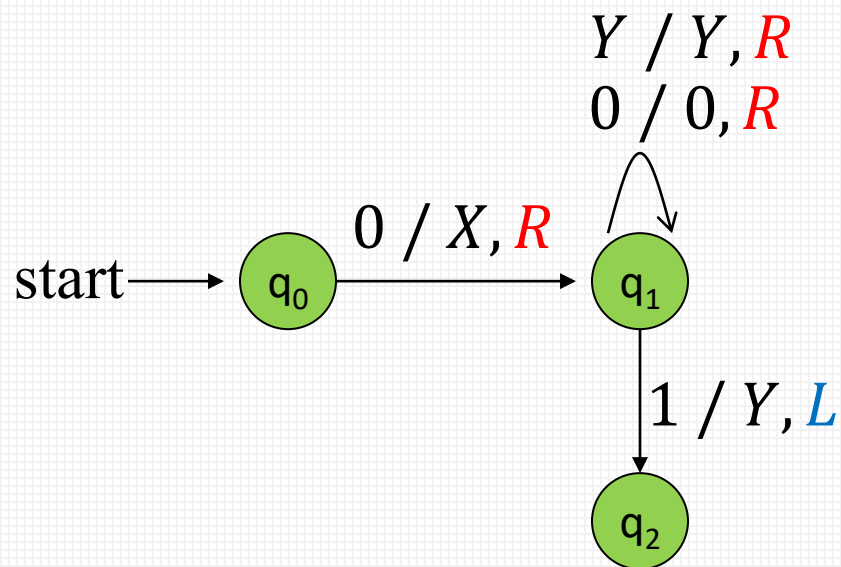


1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动

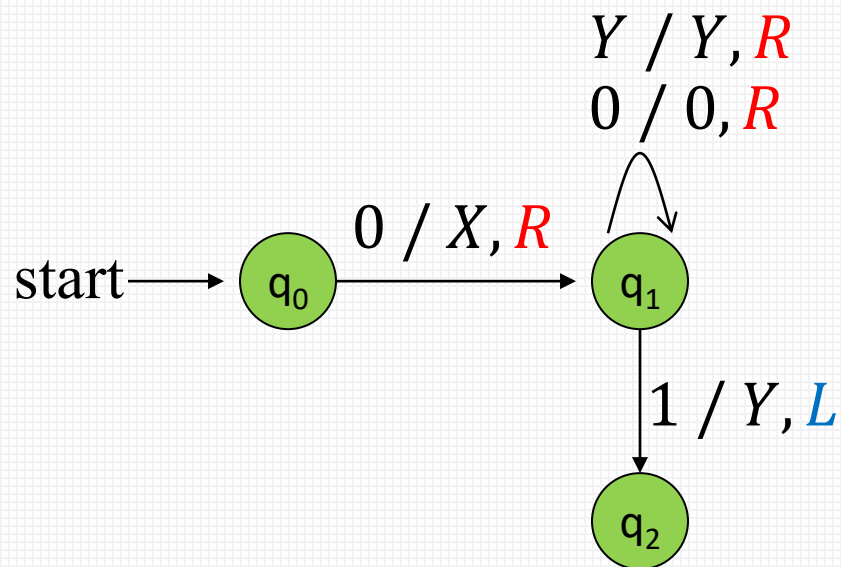
识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机



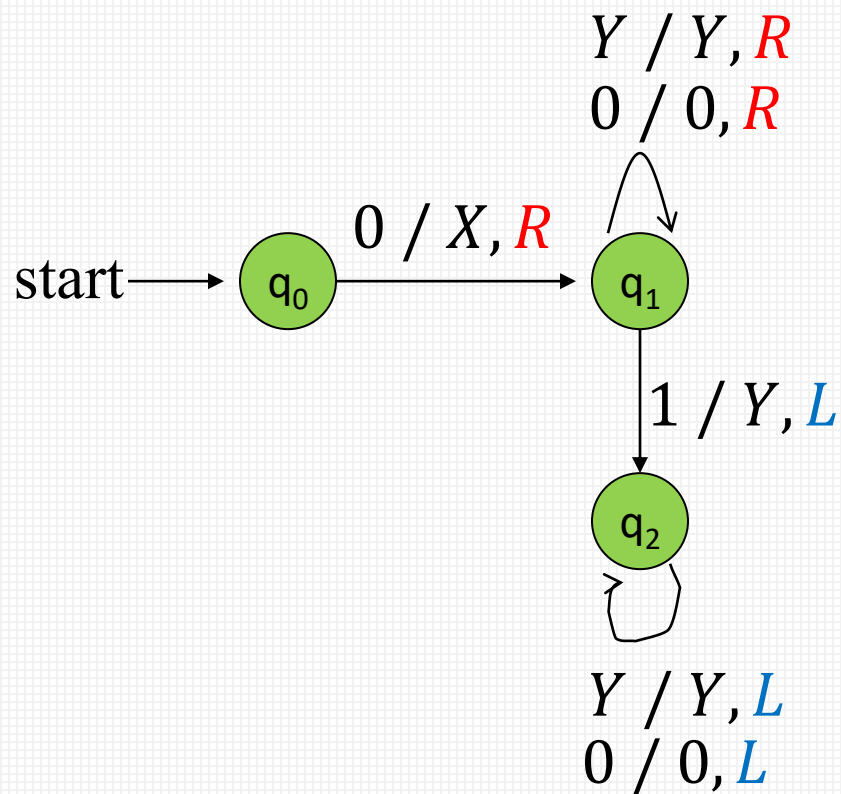
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动



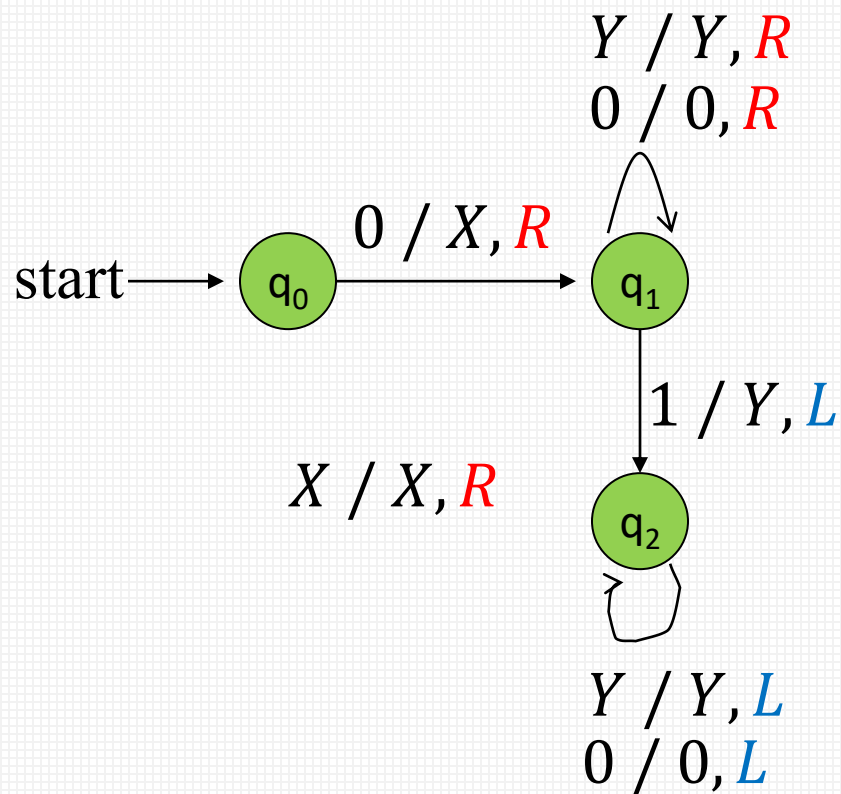
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动



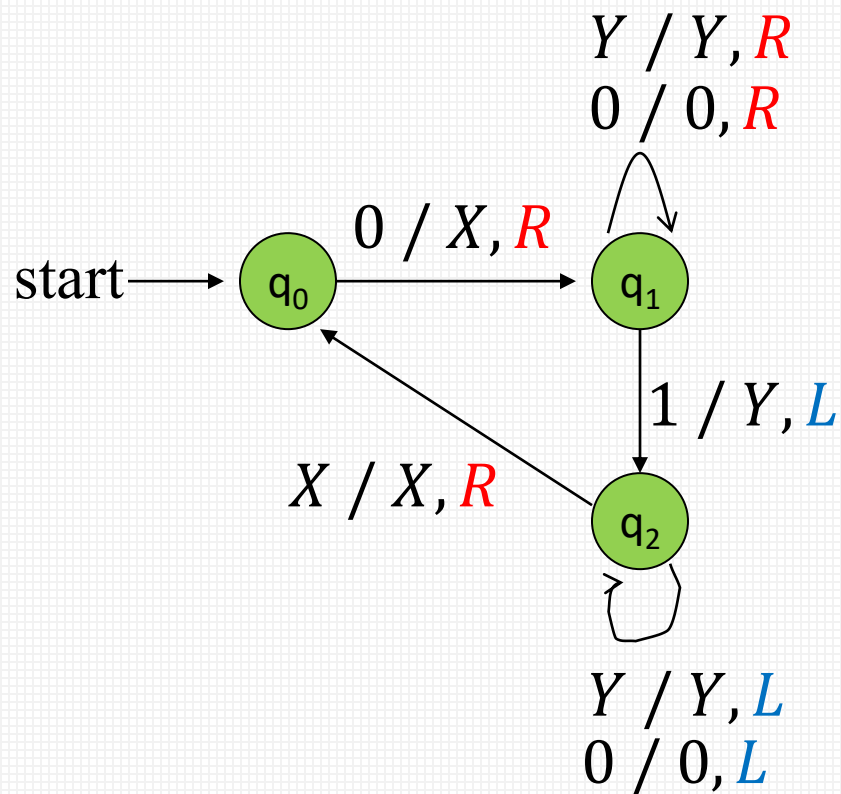
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动



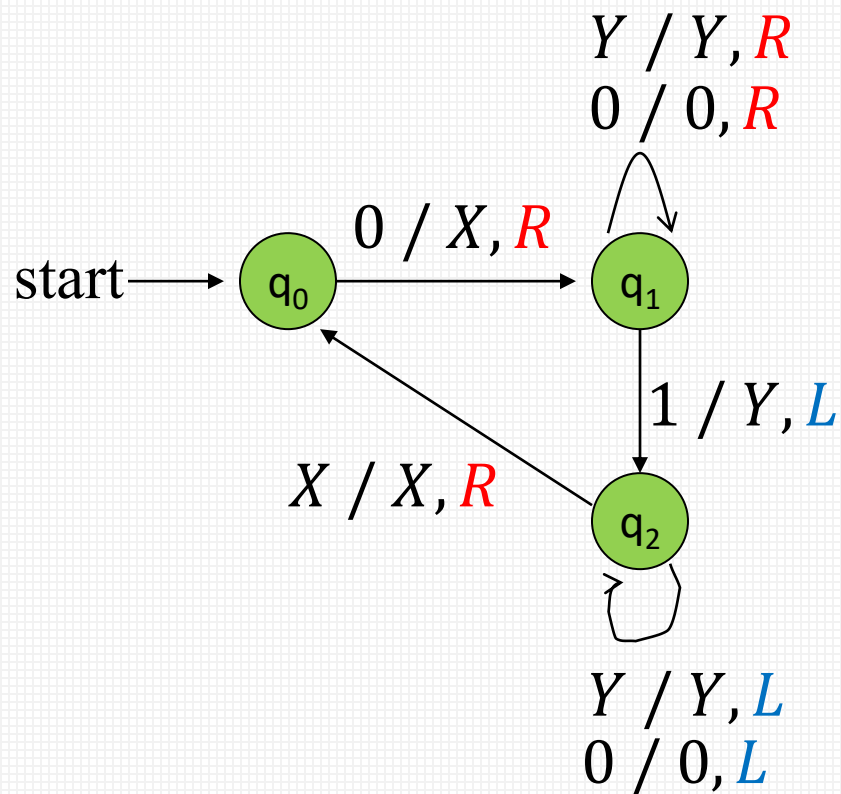
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动



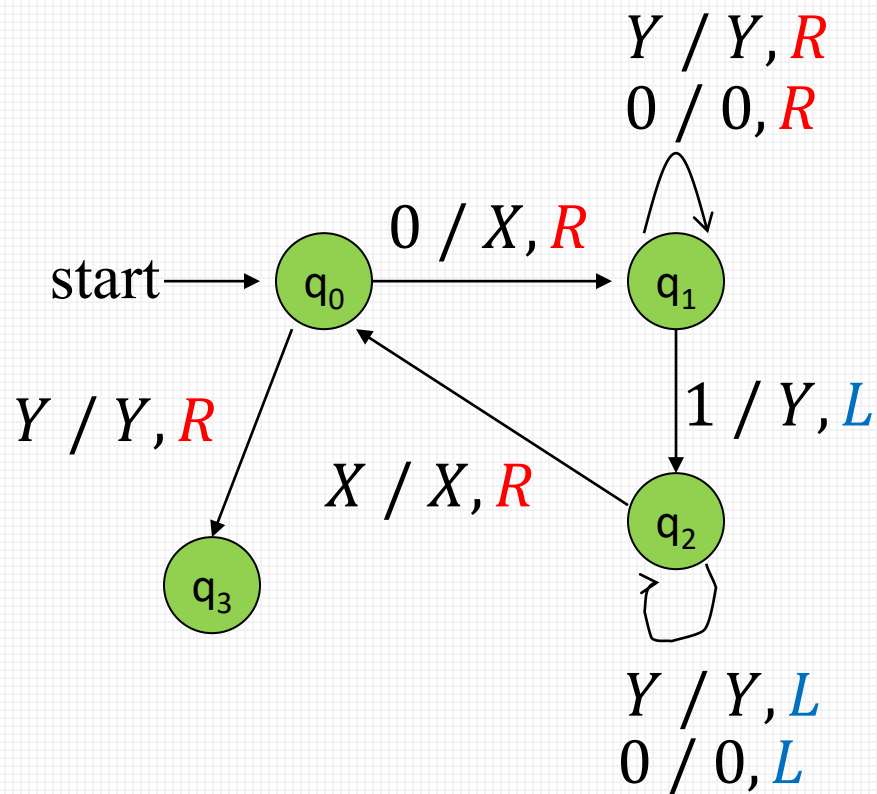
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动



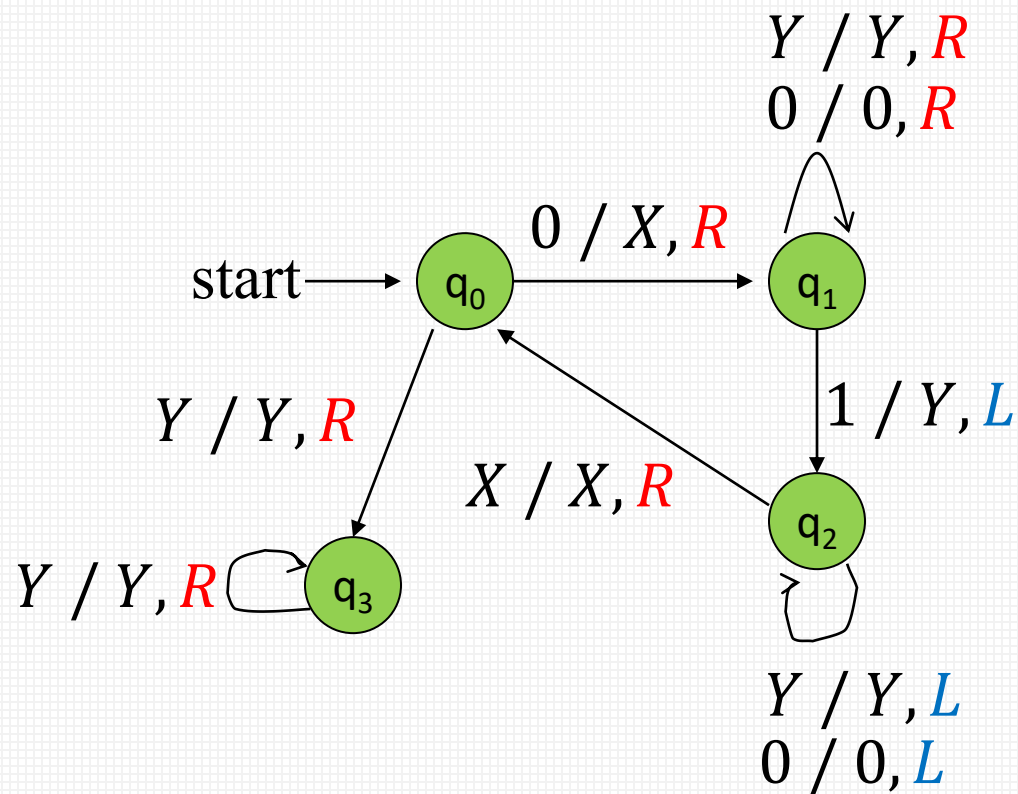
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动



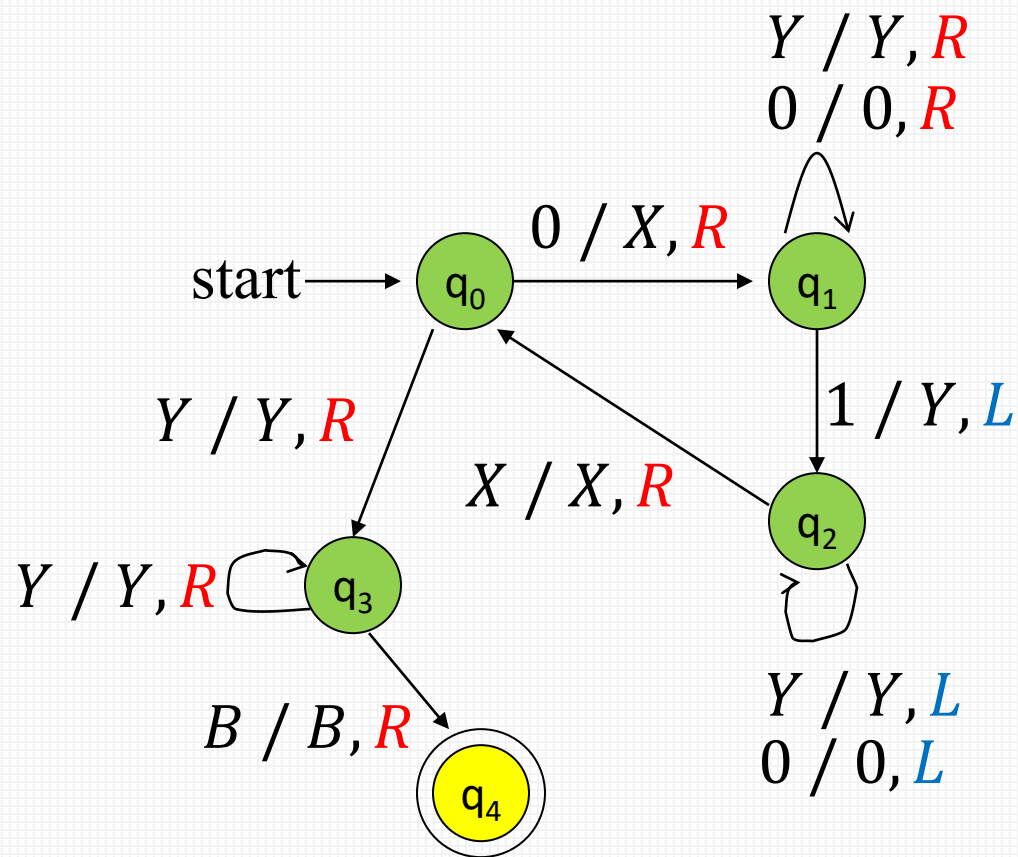
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动
4. 如果下一个要读入的符号是0, 则跳转到第一步; 否则, 向右移动检查是否还有剩余的1。如果后面已经全是Y, 那么在空格符停下, 进入接受状态。



1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动
4. 如果下一个要读入的符号是0, 则跳转到第一步; 否则, 向右移动检查是否还有剩余的1。如果后面已经全是Y, 那么在空格符停下, 进入接受状态。



1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动
4. 如果下一个要读入的符号是0, 则跳转到第一步; 否则, 向右移动检查是否还有剩余的1。如果后面已经全是Y, 那么在空格符停下, 进入接受状态。



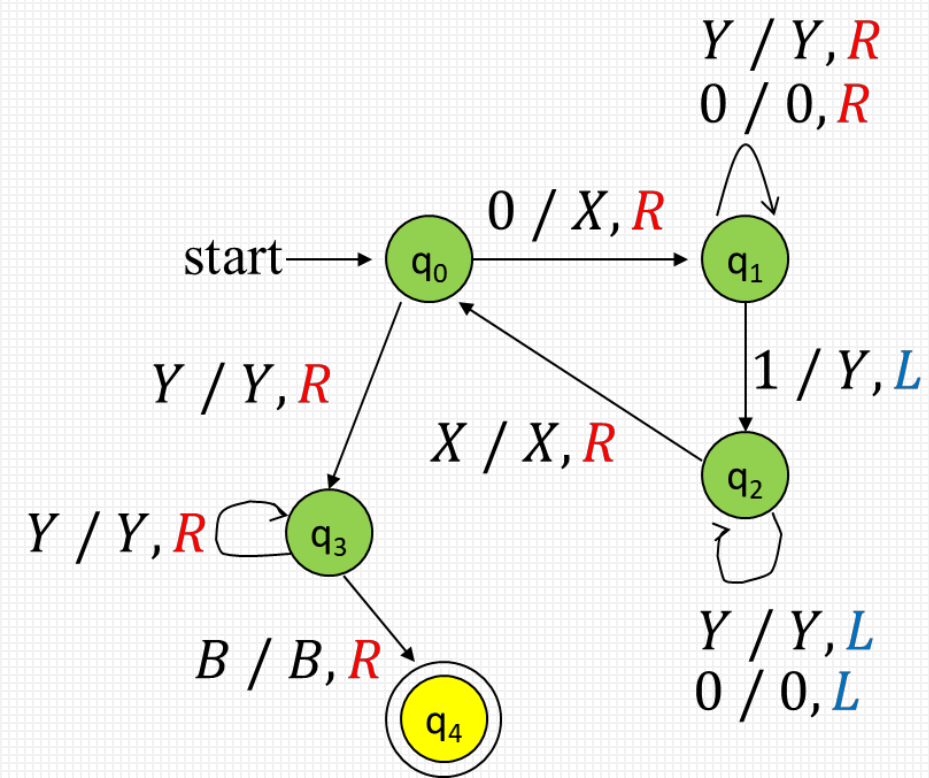
1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动
4. 如果下一个要读入的符号是0, 则跳转到第一步; 否则, 向右移动检查是否还有剩余的1。如果后面已经全是Y, 那么在空格符停下, 进入接受状态。

识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机

- 七元组定义: $M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta, q_0, B, \{q_4\})$
- 状态转移表:

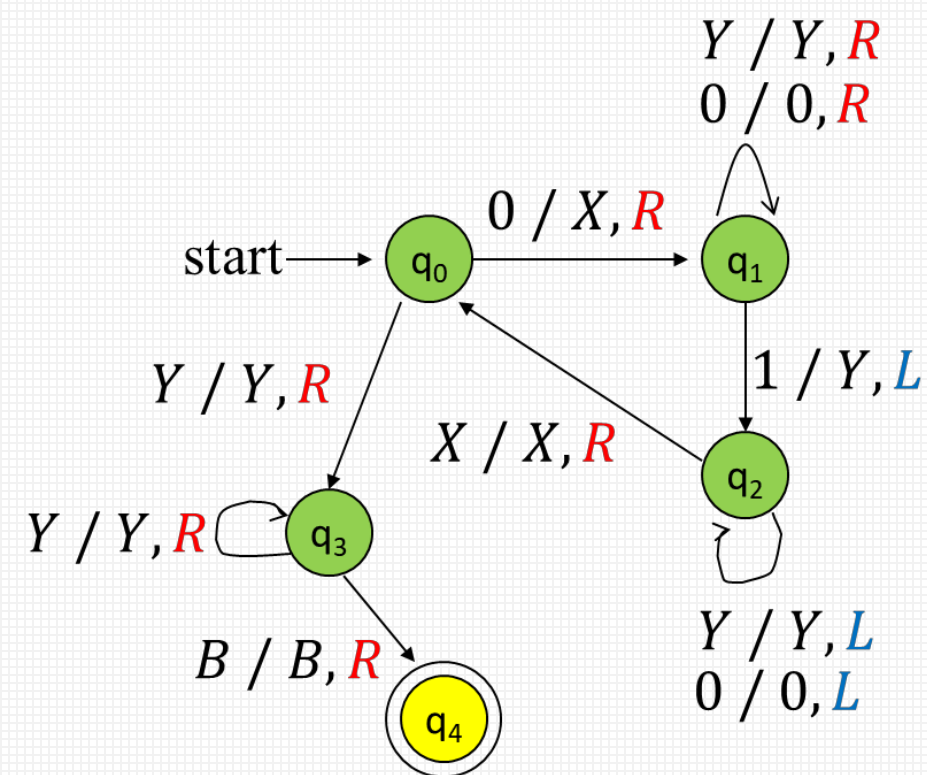
δ	0	1	X	Y	B
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, R)
q_4	—	—	—	—	—

- 接受0011的ID序列:



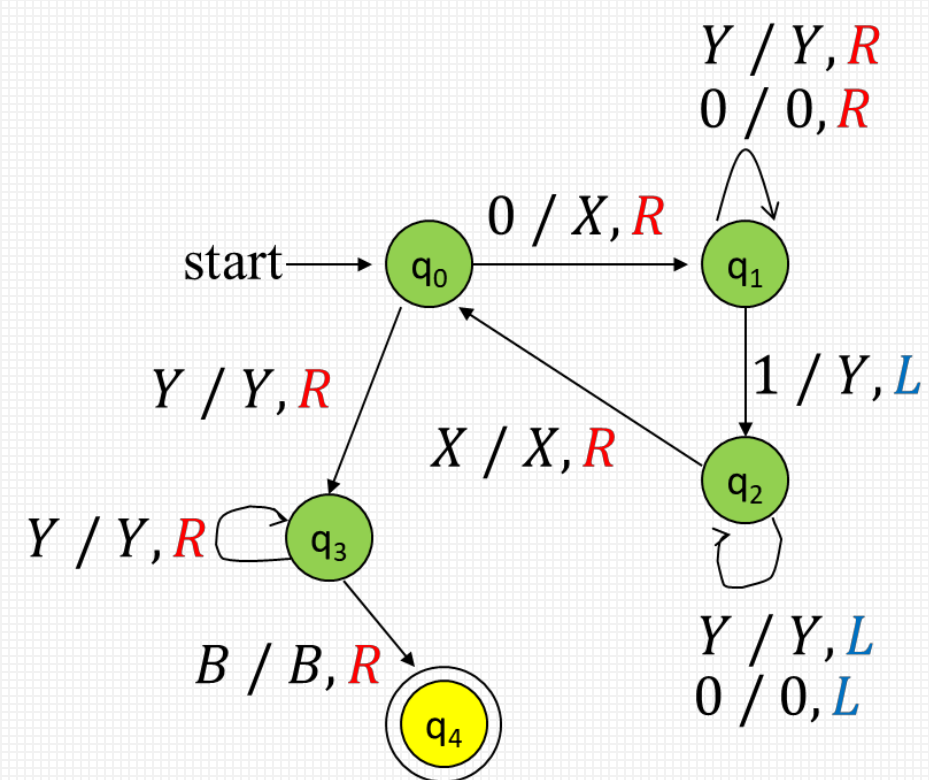
$q_0 0011 \vdash$

- 接受0011的ID序列:



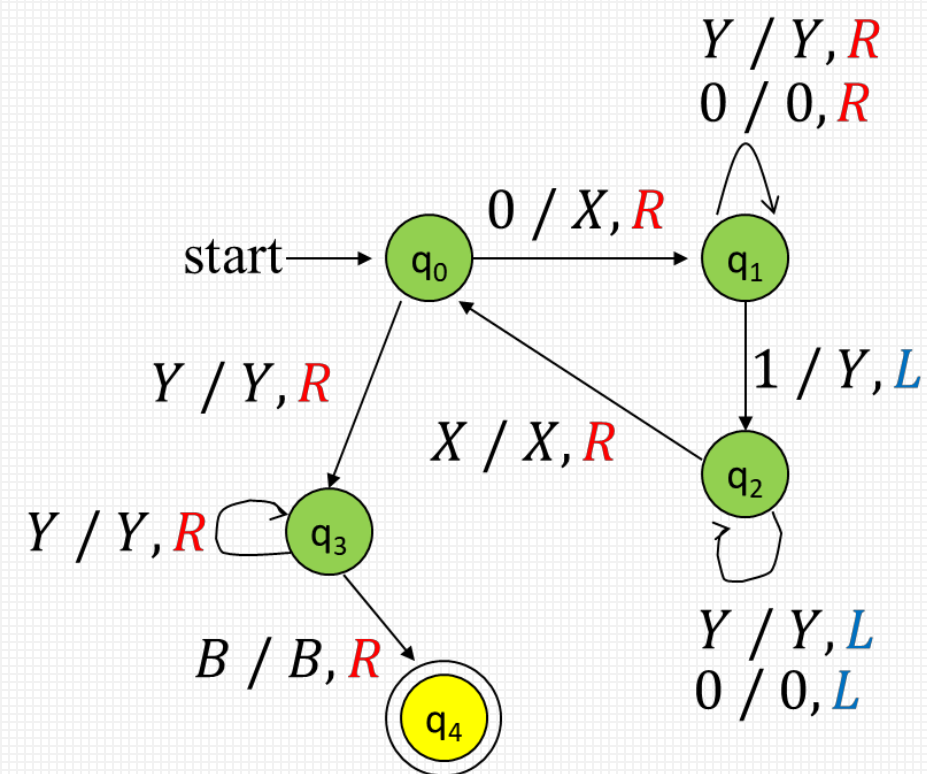
$q_0 0011 \vdash Xq_1 011 \vdash X0q_1 11 \vdash Xq_2 0Y1$
 $\vdash q_2 X0Y1 \vdash Xq_0 0Y1 \vdash XXq_1 Y1 \vdash XXYq_1 1$
 $\vdash XXq_2 YY \vdash Xq_2 XYY \vdash XXq_0 YY \vdash XXYq_3 Y$
 $\vdash XXYq_3 B \vdash XXYq_3 Bq_4 B$

- 输入0010的ID序列:



$q_0 0010 \vdash$

- 输入0010的ID序列:

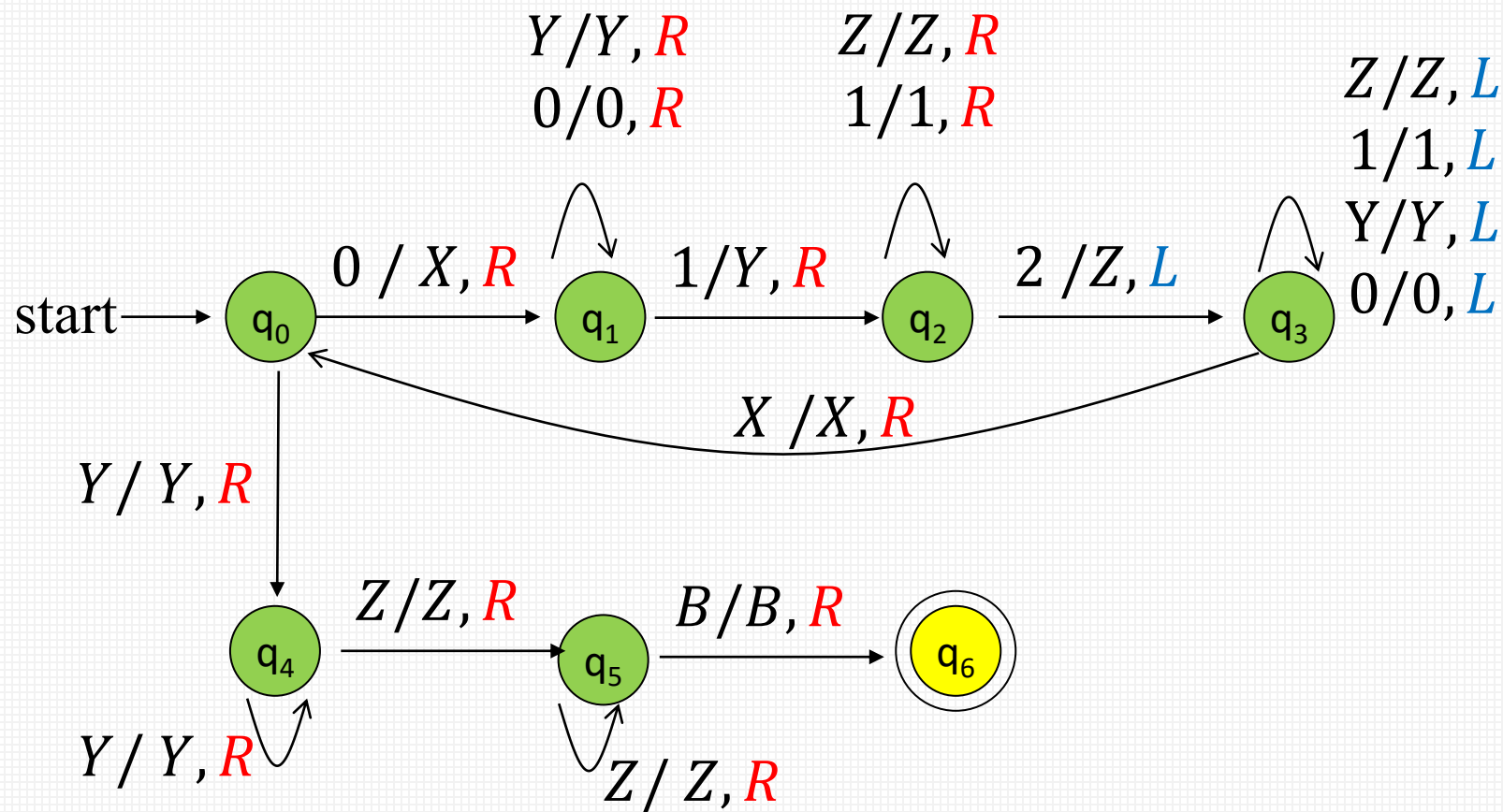


$q_0 0010 \vdash Xq_1 010 \vdash X0q_1 10 \vdash Xq_2 0Y0$
 $\vdash q_2 X0Y0 \vdash Xq_0 0Y0 \vdash XXq_1 Y0 \vdash XXYq_1 0$
 $\vdash XXY0q_1 B$

最终处于状态 q_1 ,指向空格符号,此时没有相应的动作定义。图灵机停止运转。

- 例2: 设计识别 $\{0^n 1^n 2^n | n \geq 1\}$ 的图灵机。
- 思路:
 - 首先用 X 替换最左边的 0 ，然后右移至最左边的 1 用 Y 替换之，右移至最左边的 2 用 Z 替换之。左移寻找最右的 X （以确定下一个要标记的 0 的位置），然后右移一个单元找到下一个最左的 0 ，重复循环。

- 例2: 设计识别 $\{0^n 1^n 2^n | n \geq 1\}$ 的图灵机。



- 图灵机简介、形式化定义
- **基于图灵机的整数函数运算**
- 图灵机的表示能力：语言、停机
- 扩展图灵机
- 受限图灵机

- 整数计算器：图灵机可以作为语言的识别器，也可以用作整数函数计算器。
- 例如，将整数 $i \geq 0$ 表示为字符串 0^i
- 若计算 k 个自变量 $i_1, i_2, \dots, i_k$ 的函数 f ，用 $0^{i_1} 1 0^{i_2} \dots 1 0^{i_k}$

作为图灵机 M 的输入

- 当 M 停机且输入带上为 0^m ，表示计算 $f(i_1, i_2, \dots, i_k) = m$

- 例1：设计正整数加法的图灵机

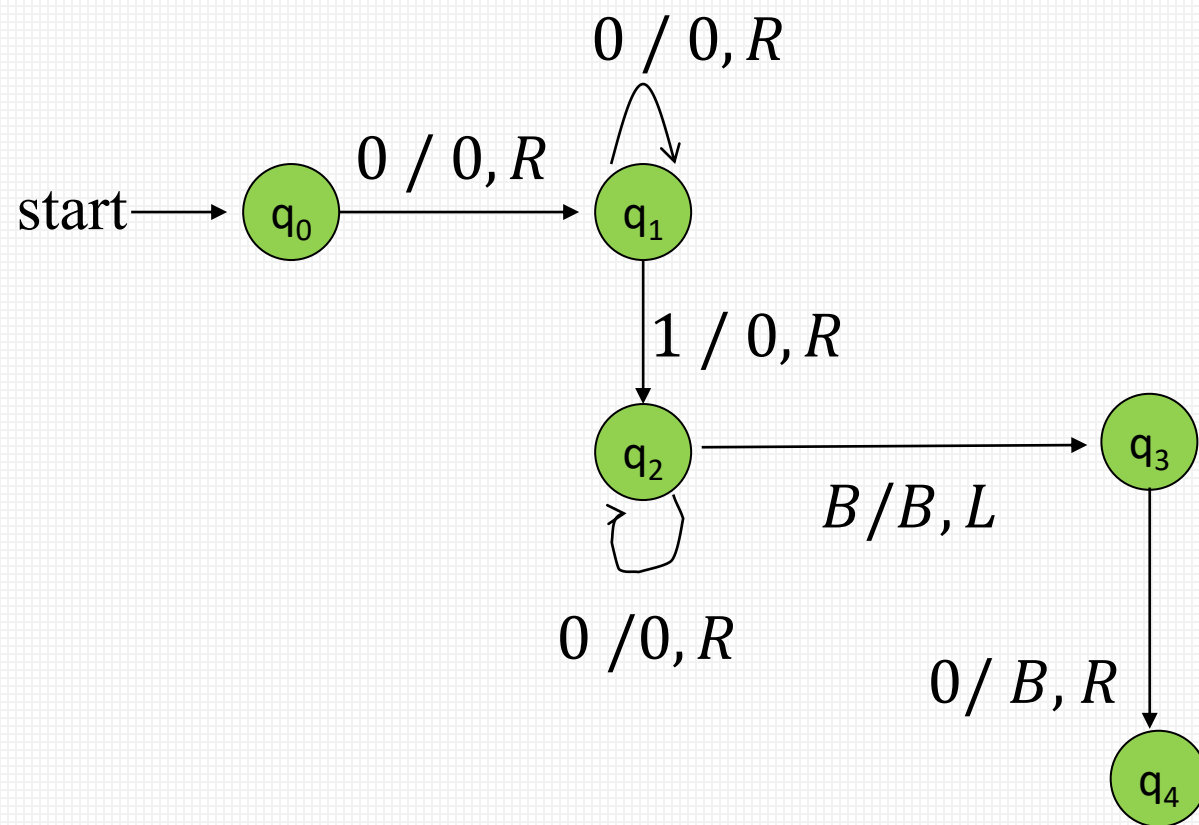
$$m + n$$

- 思路：

- 开始时，带上符号为 $0^m 1 0^n$ ，结束时，带上的符号为 $(m+n)$ 个0。
- 从最左的0开始，往右移动，跳过0，遇到分隔符1时，把1改为0（此时带上为 $m+n+1$ 个0），然后继续往右移动，跳过0，直到遇到空格符号后，将带头往左移，把带头指向的最后一个0改为空格符号，此时带上即有 $m+n$ 个0。

- 例1：设计正整数加法的图灵机

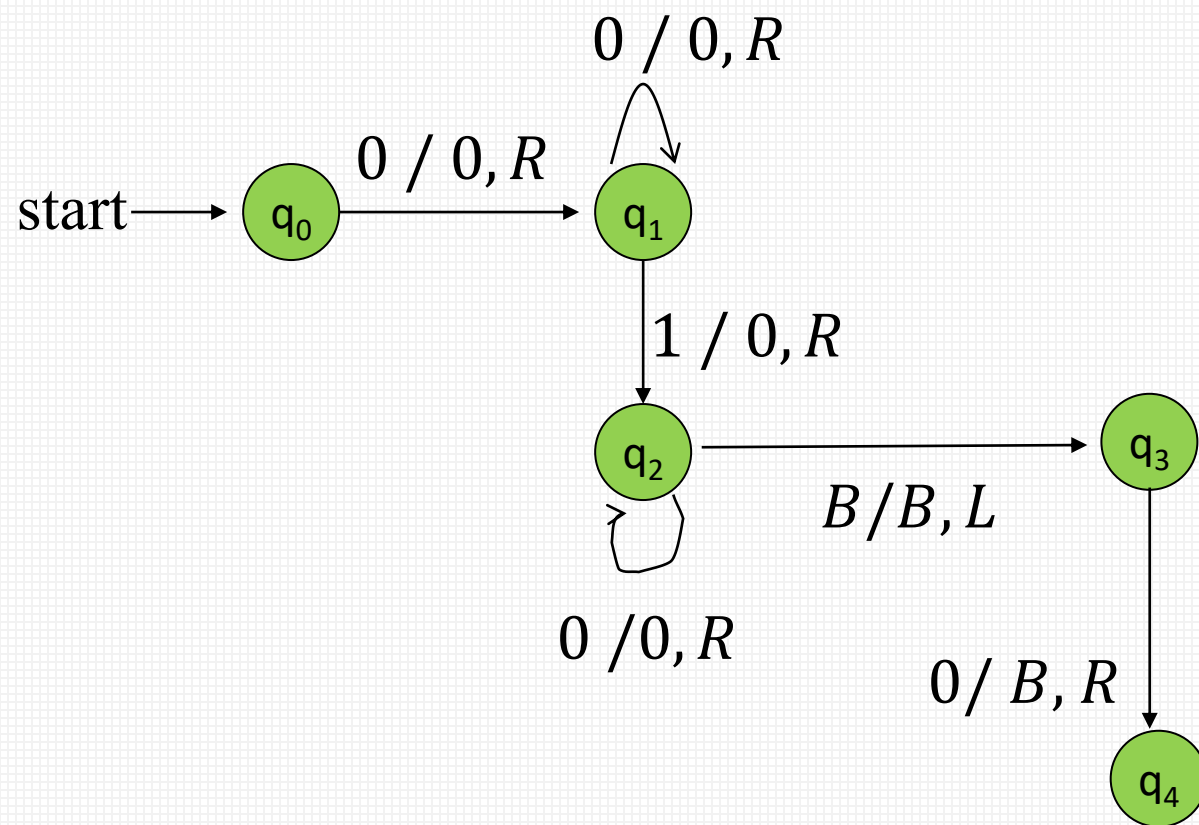
$$m + n$$



计算 $1 + 2$:
 $q_0 0 \textcolor{red}{1} 00 \vdash$

• 例1：设计正整数加法的图灵机

$m + n$



计算 $1 + 2$:

$q_0 0 \mathbf{1} 0 0 \vdash 0 q_1 \mathbf{1} 0 0 \vdash 0 0 \mathbf{1} q_2 0 0$
 $\vdash 0 0 \mathbf{0} q_2 0 \vdash 0 0 \mathbf{0} 0 q_2 B$
 $\vdash 0 0 \mathbf{0} q_3 0 \vdash 0 0 \mathbf{0} B q_4 B$

• 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, m \geq n \\ 0, m < n \end{cases}$$

• 思路：

- 开始时，带上符号为 $0^m 1 0^n$ ，结束时，带上的符号为 0（此时 0 的个数即为对应 $m \geq n$ 的结果）或者空格符（此时结果为 0，对应 $m < n$ ）。
- 重复以下循环：找到最左边的 0，将其改为 B，然后往右找到 1，继续往右，找到 0 的话把它改为 1。然后往左移动，寻找下一个最左的 0，此时得先向左找到空格符号 B，以确定下一个最左边的 0 的位置。
- 重复以上循环直到以下两种情况结束：
 - (1) 1 右边的 n 个 0 全部改为了 1，向右移遇到了空格符，说明此时 $0^m 1 0^n$ 中右边的 n 个 0 已经都被改为了 1，而此时左边已将 $(n+1)$ 个 0 改为了 B（多改了 1 个）。带上此时有 $n+1$ 个 1。把这 $n+1$ 个 1 改为 B，把左边多改的 0 还原为 0。这样带上剩下 $m-n$ 个 0（由 $m-(n+1)+1=m-n$ ）。
 - (2) 向左已经找不到 0（即 m 个 0 已经都改为了 B），此时把带上剩下的 1 和 0 全部变为空格符号，留下一个只有空格符的带。

- 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

- 思路分析：以 $m = 3, n = 2$ 为例

...	B	0	0	0	1	0	0	B	...
...	B	B	0	0	1	0	0	B	...
...	B	B	0	0	1	1	0	B	...
...	B	B	B	0	1	1	1	B	...
...	B	B	B	B	1	1	1	B	...
...	B	B	B	0	B	B	B	B	...

每当在1的左边将一个0改变为B，就在1的右边将一个0改为1。重复以上循环，直到以下两种情况结束：

(1) 1右边的 n 个0全部改为了1，向右移遇到了空格符，说明此时 $0^m 10^n$ 中右边的 n 个0已经都被改为了1，而此时左边已将 $(n+1)$ 个0改为了B（多改了1个）。带上此时有 $n+1$ 个1。把这 $n+1$ 个1改为B，把左边多改的0还原为0。这样带上剩下 $m - n$ 个0(由 $m - (n + 1) + 1 = m - n$)。

- 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

- 思路分析：以 $m = 2, n = 3$ 为例

...	B	0	0	1	0	0	0	B	...
...	B	B	0	1	0	0	0	B	...
...	B	B	0	1	1	0	0	B	...
...	B	B	B	1	1	1	0	B	...
...	B	B	B	B	B	B	B	B	...

每当在1的左边将一个0改变为B，就在1的右边将一个0改为1。重复以上循环，直到以下两种情况结束：

(2) 向左已经找不到0（即 m 个0已经都改为了B），此时把带上剩下的1和0全部变为空格符号，留下一个只有空格符的带。

• 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

往左找不到0，
已标记完

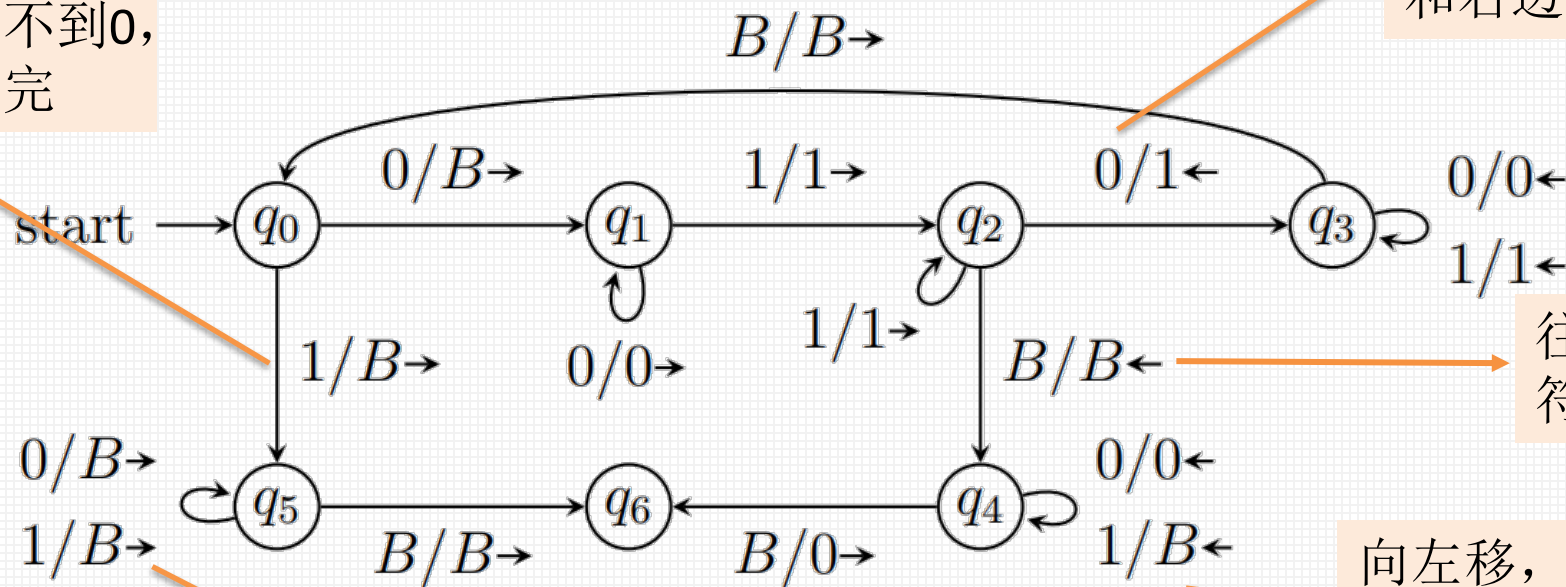
正常的标记标记左边的0为空格符
和右边匹配的0为1.

往右找不到0，遇到空格
符号。

向左移，把1改为空格符，不
改变左边未标记过的0

把左边多改的0进行还原

把带上剩下的1和0全部变为空
格符号。

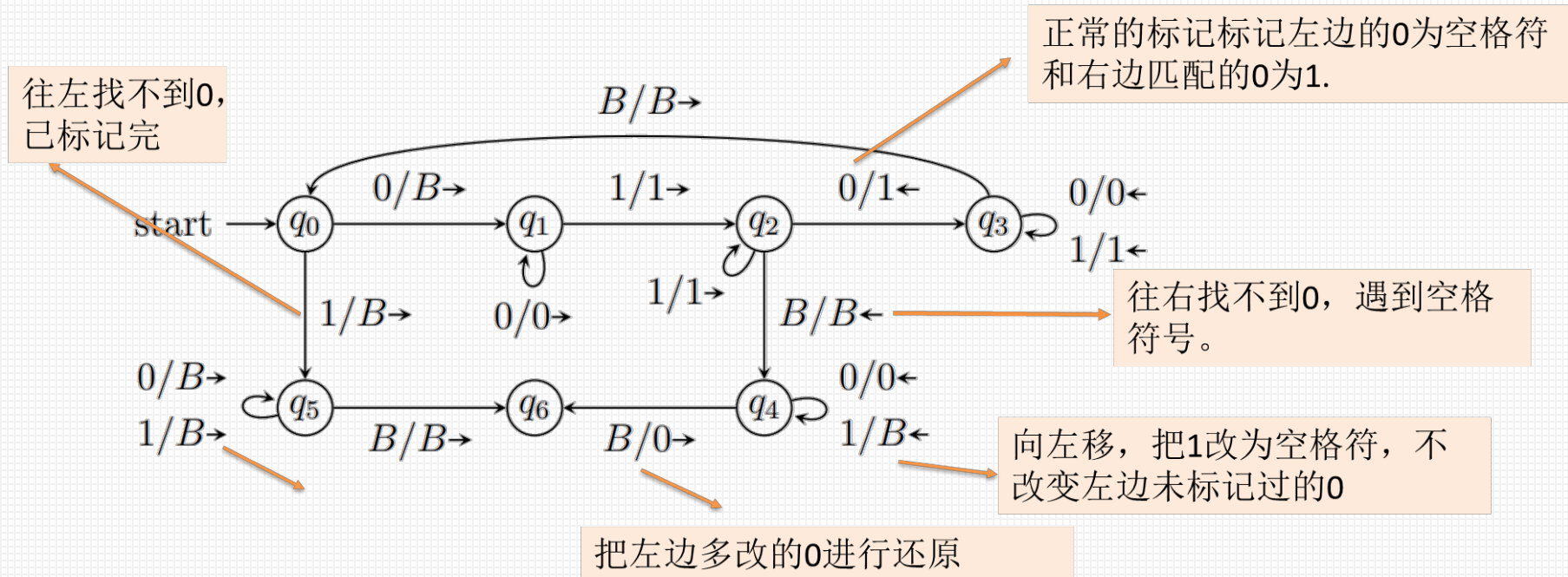


• 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

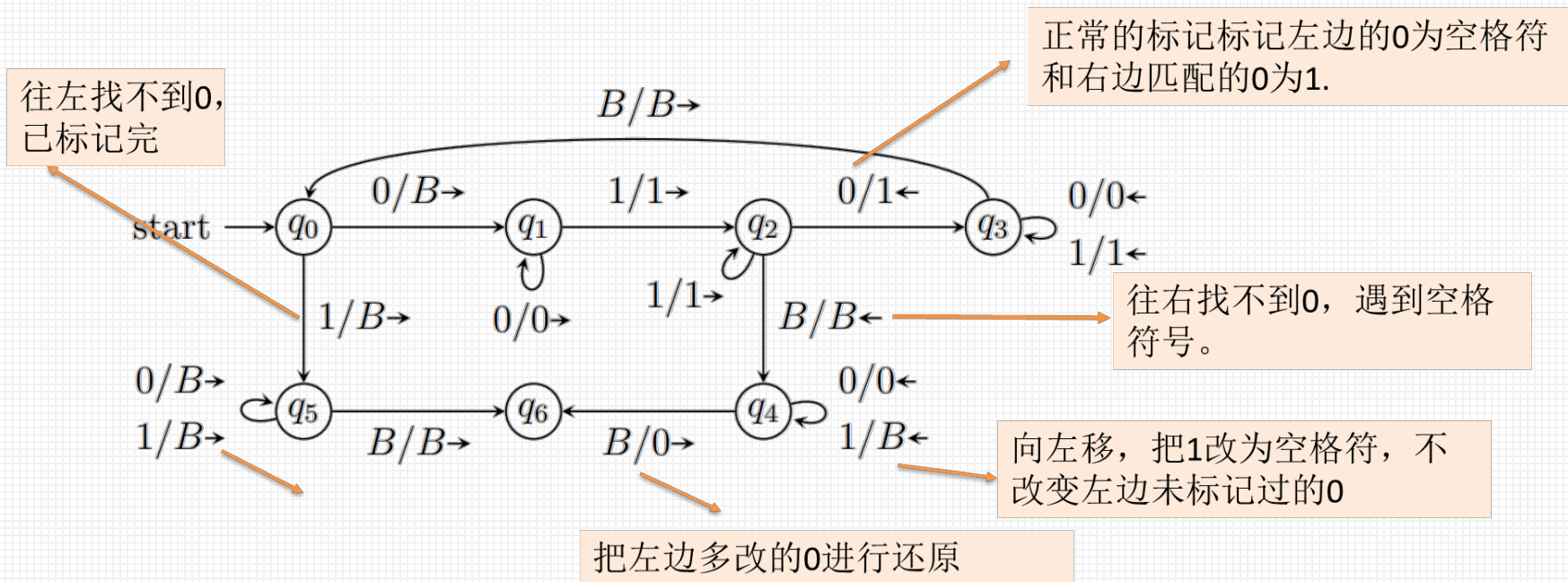
计算 $1 \dot{-} 2$:

$q_0 0 \mathbf{1} 0 0 \vdash q_1 \mathbf{1} 0 0 \vdash \mathbf{1} q_2 0 0$
 $\vdash q_3 \mathbf{1} 1 0 \vdash q_3 B \mathbf{1} 1 0 \vdash q_0 \mathbf{1} 1 0$
 $\vdash q_5 1 0 \vdash q_5 0 \vdash q_5 B \vdash q_6 B$



• 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

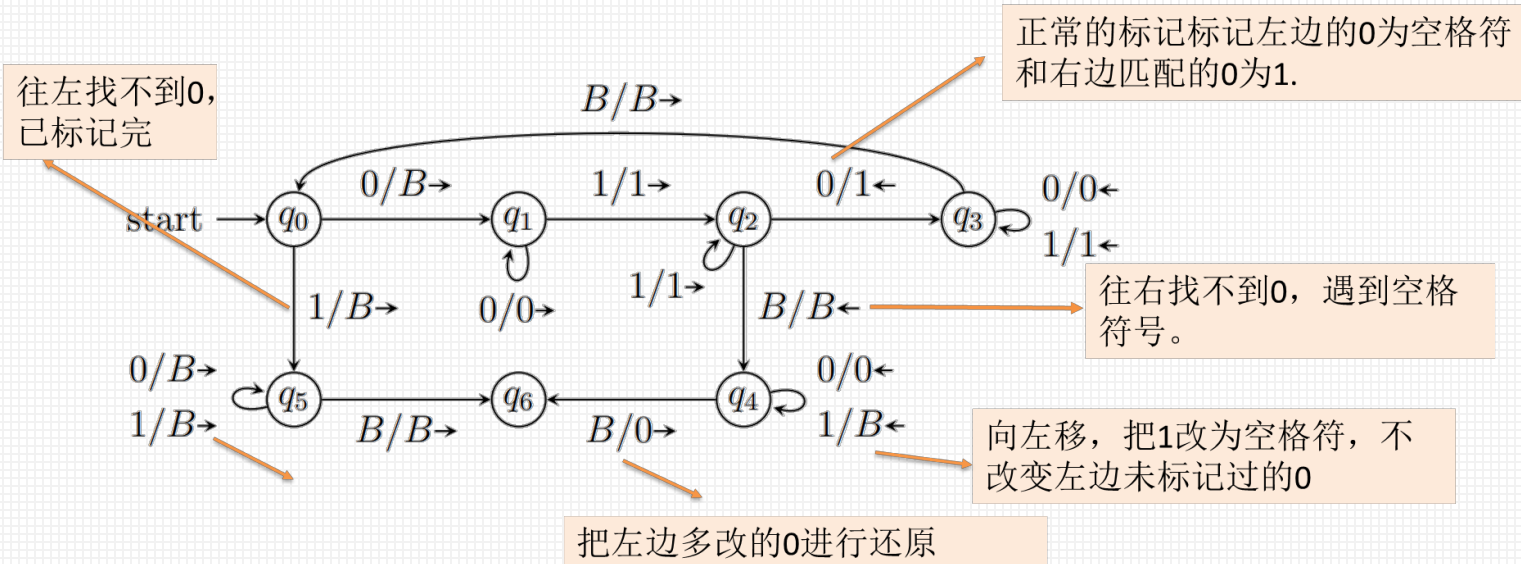


计算 $2 \dot{-} 1$:

$q_0 00\mathbf{1}0 \vdash q_1 0\mathbf{1}0 \vdash q_1 0\mathbf{1}0$
 $\vdash 0q_1\mathbf{1}0 \vdash 0\mathbf{1}q_2 0 \vdash 0q_3\mathbf{1}1$
 $\vdash q_3 0\mathbf{1}1 \vdash q_3 B0\mathbf{1}1 \vdash q_0 0\mathbf{1}1$
 $\vdash q_1\mathbf{1}1 \vdash \mathbf{1}q_2 1 \vdash \mathbf{1}1q_2 B$
 $\vdash \mathbf{1}q_4 1 \vdash q_4\mathbf{1} \vdash q_4 B \vdash 0q_6 B$

• 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \div n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$



计算 $3 \div 2$:

$q_0 000100 \vdash q_1 00100$
 $\vdash 0q_1 0100 \vdash 00q_1 100$
 $\vdash 001q_2 00 \vdash 00q_3 110$
 $\vdash 0q_3 0110 \vdash q_3 00110$
 $\vdash q_3 B00110 \vdash q_0 00110$
 $\vdash q_1 0110 \vdash 0q_1 110 \vdash 01q_2 10$
 $\vdash 011q_2 0 \vdash 01q_3 11$
 $\vdash 0q_3 111 \vdash q_3 0111$
 $\vdash q_3 B0111 \vdash q_0 0111$
 $\vdash q_1 111 \vdash 1q_2 11 \vdash 11q_2 1$
 $\vdash 111q_2 B \vdash 11q_4 1 \vdash 1q_4 1$
 $\vdash q_4 1 \vdash q_4 B \vdash 0q_6 B$

- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- **图灵机的表示能力：语言、停机**
- 扩展图灵机
- 受限图灵机

- **定义：**如果 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ 是一个图灵机, 则 M 接受的语言为

$$L(M) = \{w \mid w \in \Sigma^*, q_0 w \vdash^* \alpha p \beta, p \in F, \alpha, \beta \in \Gamma^*\}$$

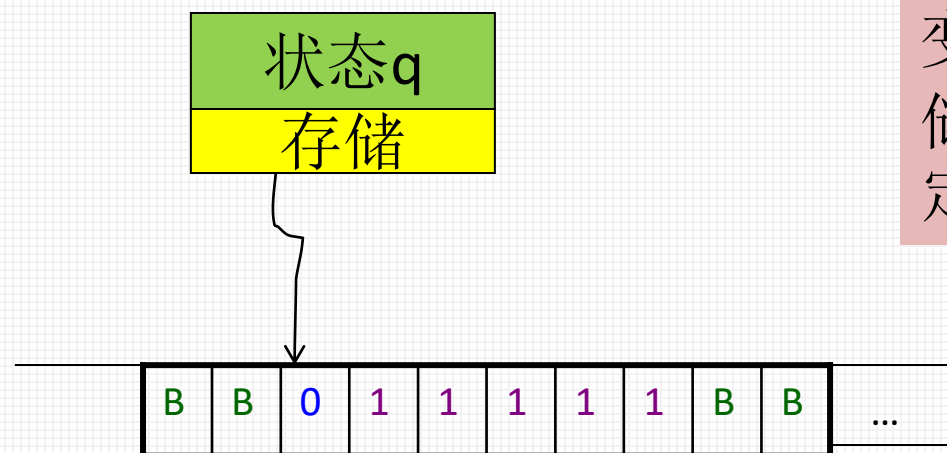
- 初始时, 输入串放在输入带上, M 处于 q_0 , 带头位于输入串的第一个字符上。 M 最终会进入某个终止（接受）状态。
- 图灵机所接受的语言称为**递归可枚举**(Recursively Enumerable)语言。
- 一般总是假定当输入串 w 被接受时, 即 $w \in L(M)$ 时, M 总会处于接受状态并停机 (**halt**), **即没有下一个动作的定义**。
- 而当 $w \notin L(M)$, 可能有两种情况:
 - (1) 永远不停机,
 - (2) 停机但是不处于接受状态。

- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- 图灵机的表示能力：语言、停机
- **扩展图灵机**
- 受限图灵机

- 状态中存储：有穷制器中可以存储有限个符号的图灵机：

$$M' = (Q', \Sigma, \Gamma, \delta, q_0', B, F')$$

其中 $Q' = Q \times \Gamma \times \cdots \times \Gamma$, $q_0' = [q_0, B, \cdots, B]$ 。

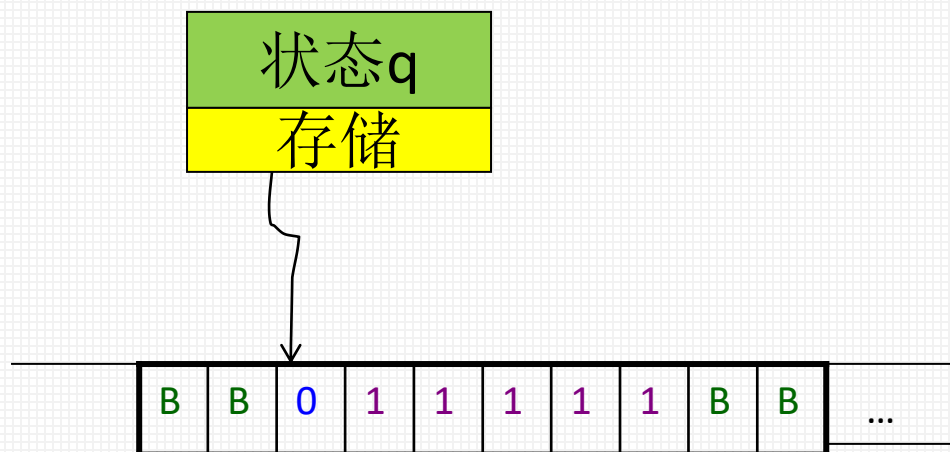


对图灵机的这种修改，只需要将有穷控制器的状态由原来的状态 q ，变为一个元组(tuple)，即状态和存储中的符号串。而初始状态可以定义为 q_0 和空格符组成的符号串。

- 状态中存储：有穷制器中可以存储有限个符号的图灵机：

$$M' = (Q', \Sigma, \Gamma, \delta, q_0', B, F')$$

其中 $Q' = Q \times \Gamma \times \cdots \times \Gamma$, $q_0' = [q_0, B, \cdots, B]$ 。

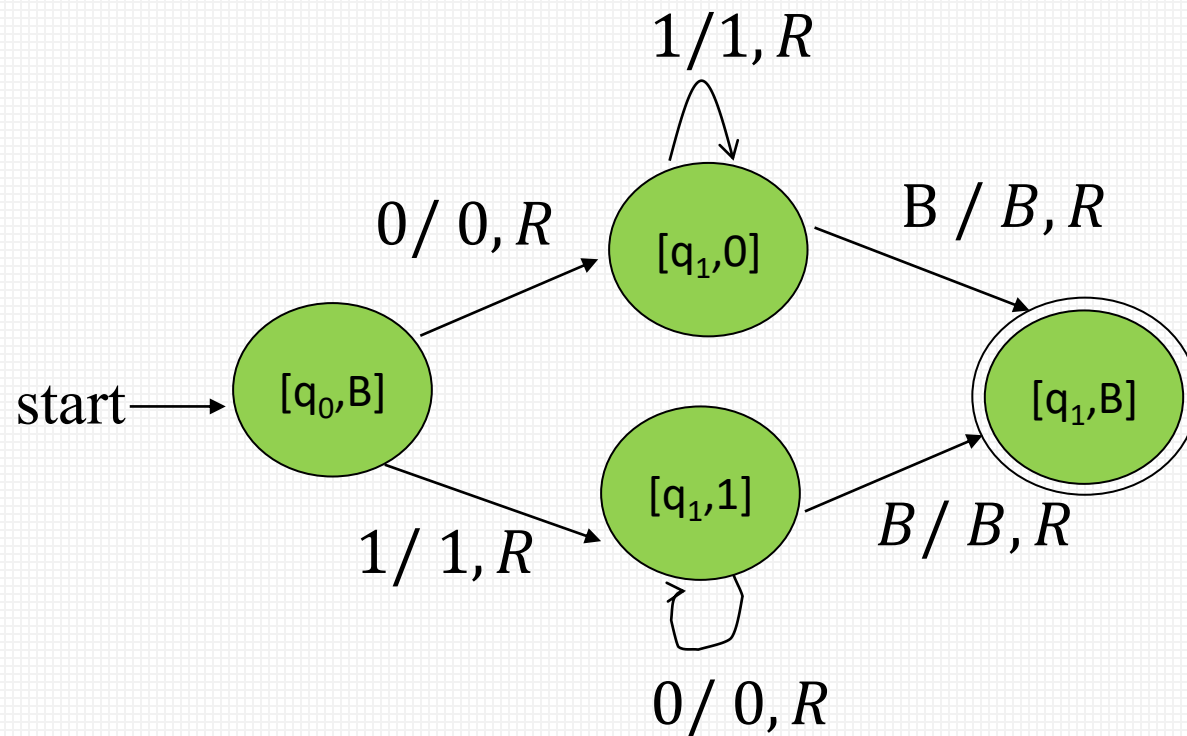


比如，若存储中的符号可为A或B，有穷控制器状态可为 q_0 或 q_1 ，则图灵机的状态可以为 $[q_0, A]$, $[q_0, B]$, $[q_1, A]$, $[q_1, B]$

- 例:利用状态中存储,设计图灵机,能够对输入的01串进行扫描识别,并要求第一个符号在后面不出现。(即识别语言 $01^* + 10^*$)
 - 图灵机 $M = (Q, \Sigma = \{0,1\}, \Gamma = \{0,1,B\}, \delta, [q_0, B], B, \{[q_1, B]\})$
 - 状态集 $Q = \{q_0, q_1\} \times \{0,1,B\}$, 即每个状态是一个二元组, 一个对应状态控制部分, 一个对应存储部分。
 - q_0 表示图灵机还没有读入第一个符号, q_1 表示已读入了第一个符号, 然后开始检查该符号在后续没有出现
 - 存储部分, 用来存储第一个符号, 初始的时候存储中为空格符B,后续可以为0或者1.

- 例:利用状态中存储, 设计图灵机, 能够对输入的01串进行扫描识别, 并要求第一个符号在后面不出现。(即识别语言 $01^* + 10^*$)
 - 图灵机 $M = (Q, \Sigma = \{0,1\}, \Gamma = \{0,1,B\}, \delta, [q_0, B], B, \{[q_1, B]\})$
 - δ 函数的定义:
 - $\delta([q_0, B], a) = ([q_1, a], a, R), a = 0 \text{ 或 } a = 1$ (初始时候, 读入存储第一个符号 a , 然后转移到状态 q_1)
 - $\delta([q_1, a], \bar{a}) = ([q_1, a], \bar{a}, R), \bar{a}$ 为 a 的补 (当后面的符号和存储中的不一样时, 继续往右移, 存储中符号不变)
 - $\delta([q_1, a], B) = ([q_1, B], B, R)$ (往右移遇到了空格符号, 进入接受状态)
 - 注意, 对于 $\delta([q_1, a], a), a = 0 \text{ 或 } a = 1$ 是没有定义的。因此, 如果后面遇到了和存储中一样的符号(即第一个符号), 便会停机且不处于接受状态。

- 例: 利用状态中存储, 设计图灵机, 能够对输入的01串进行扫描识别, 并要求第一个符号在后面不出现。(即识别语言 $01^* + 10^*$)

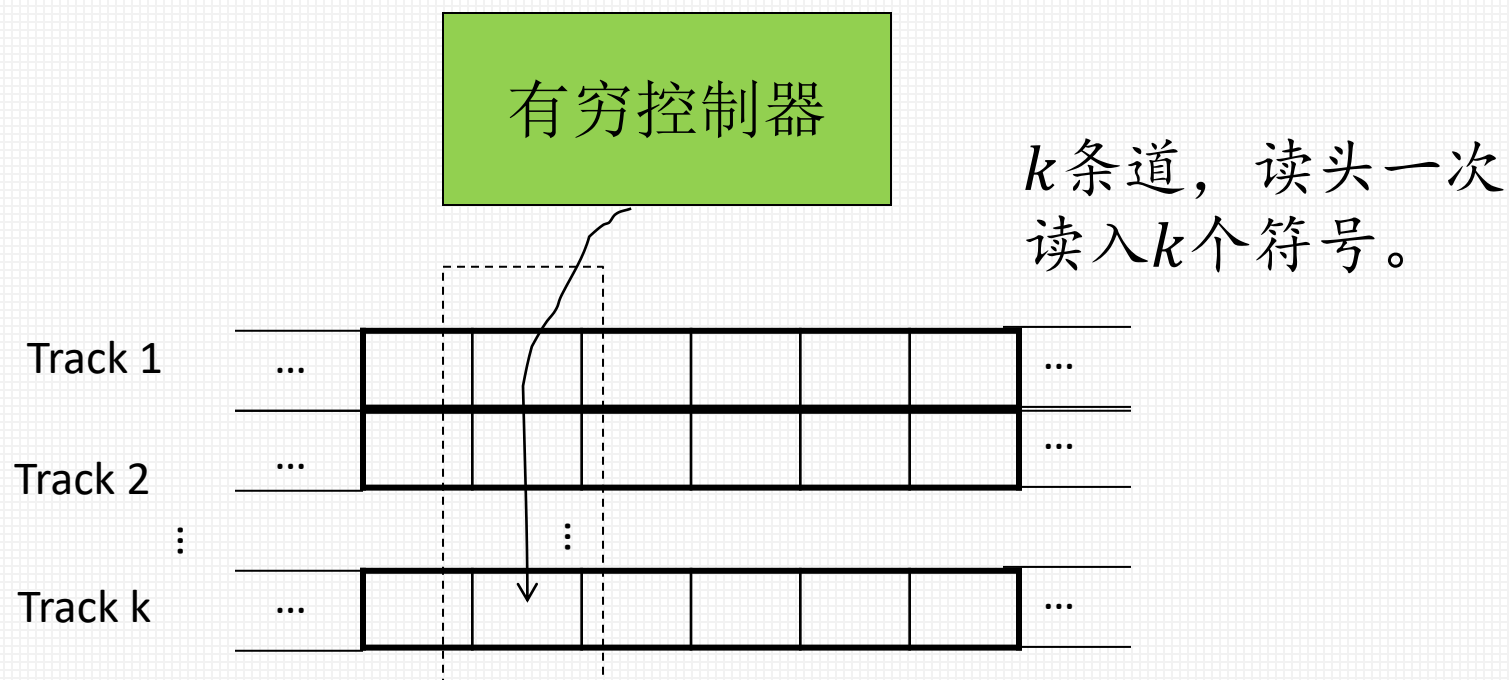


- $\delta([q_0, B], a) = ([q_1, a], a, R)$, $a = 0$ 或 $a = 1$ (初始时候, 读入存储第一个符号 a , 然后转移到状态 q_1)
- $\delta([q_1, a], \bar{a}) = ([q_1, a], \bar{a}, R)$, $\bar{a}$ 为 a 的补 (当后面的符号和存储中的不一样时, 继续往右移, 存储中符号不变)
- $\delta([q_1, a], B) = ([q_1, B], B, R)$ (往右移遇到了空格符号, 进入接受状态)

多道 (multi-track) 图灵机:

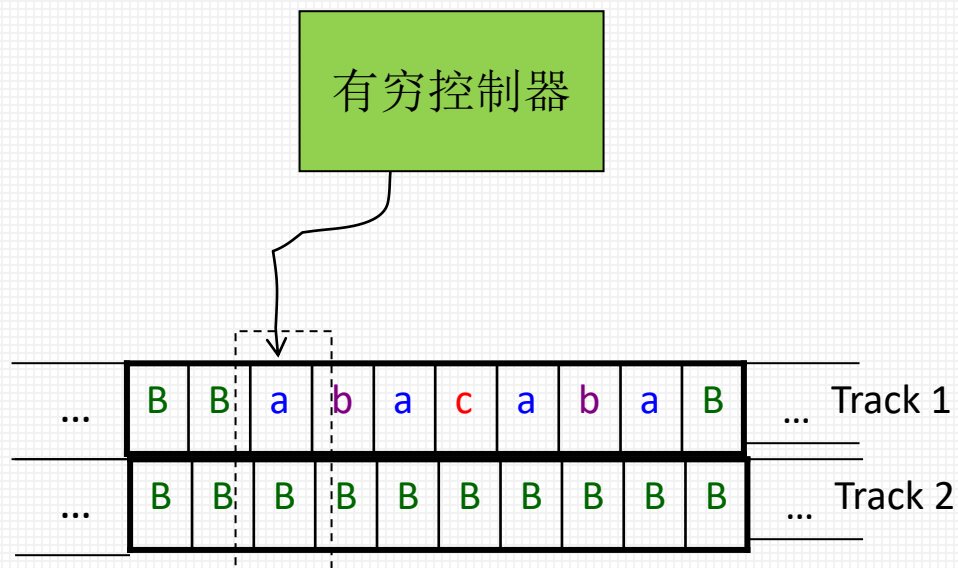
$$M' = (Q, \Sigma, \Gamma, \delta, q_0, B', F)$$

其中 $\Gamma' = \Gamma \times \Gamma \times \cdots \times \Gamma$ 。



- 例：利用状态中存储与多道设计TM识别

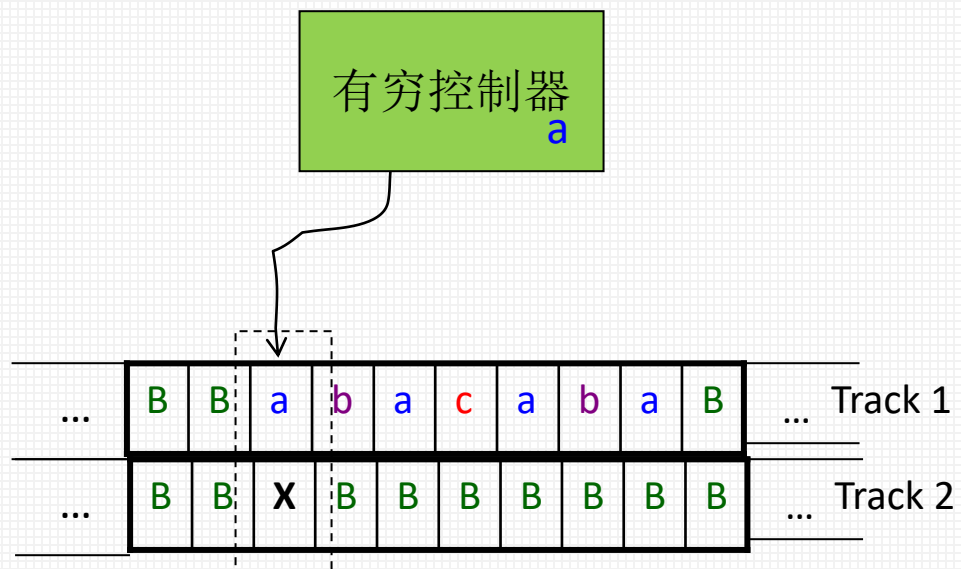
$$L = \{wcw | w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

- 例：利用状态中存储与多道设计TM识别

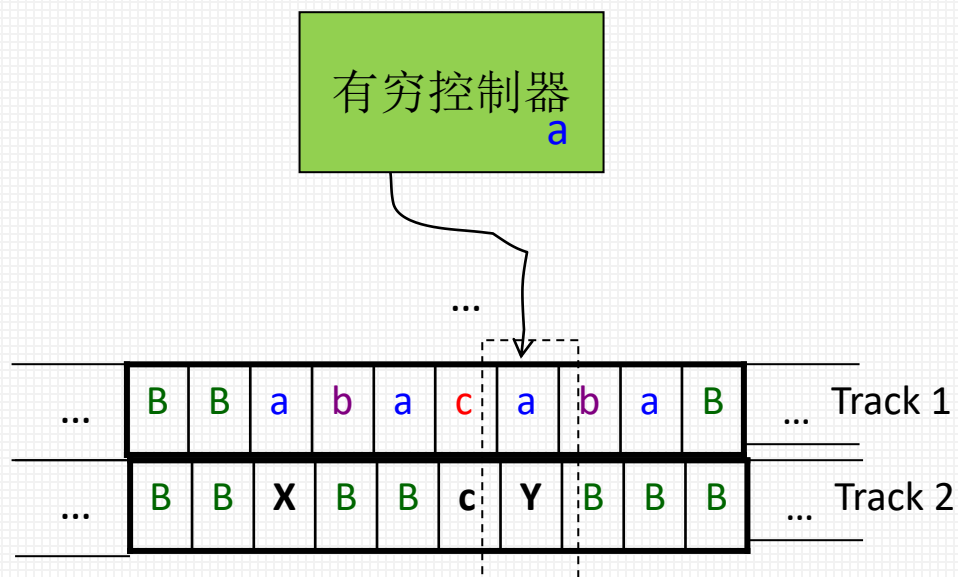
$$L = \{wcw | w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

- 例：利用状态中存储与多道设计TM识别

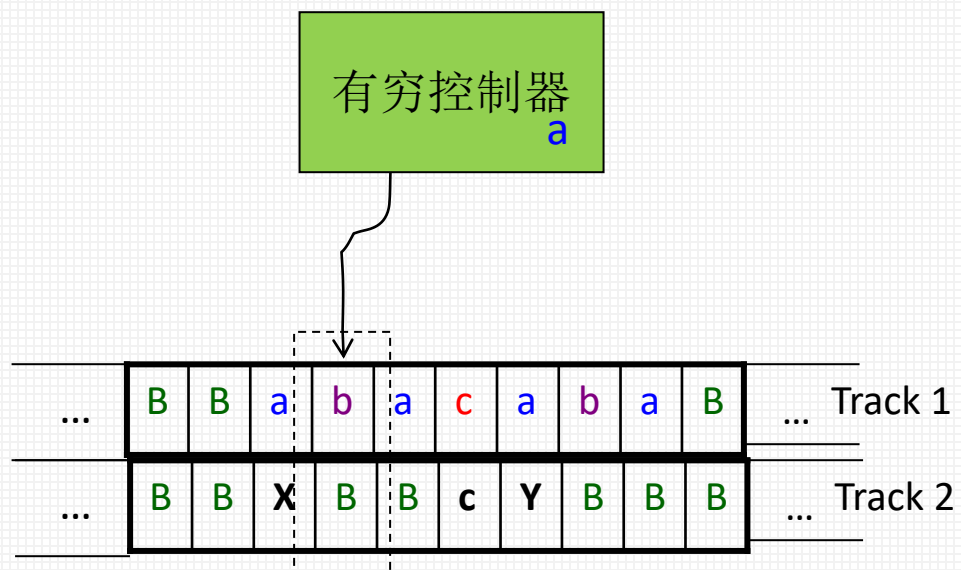
$$L = \{wcw | w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

- 例：利用状态中存储与多道设计TM识别

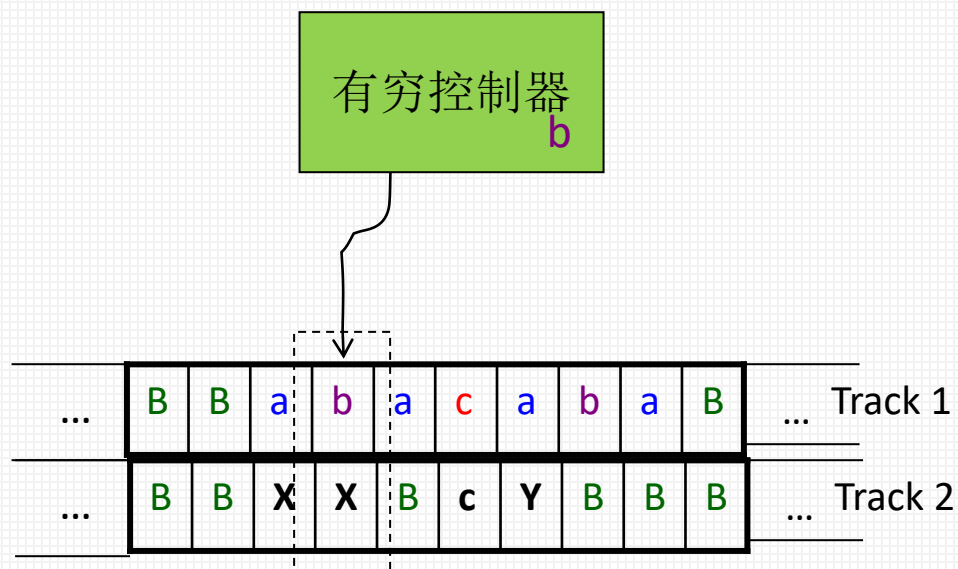
$$L = \{wcw | w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

- 例：利用状态中存储与多道设计TM识别

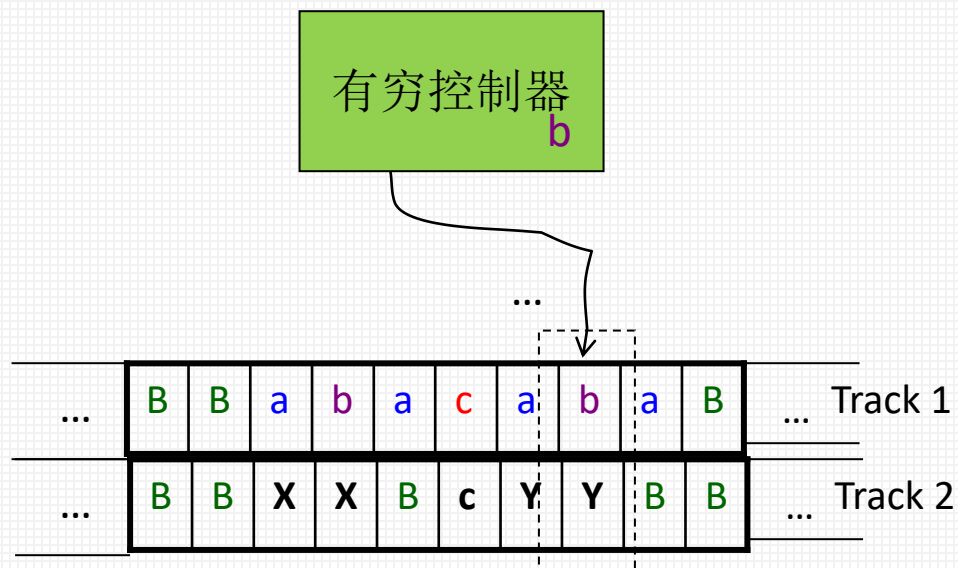
$$L = \{wcw \mid w \in \{a, b\}^*\}$$



Track 2主要起到做标记的作用。

- 例：利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

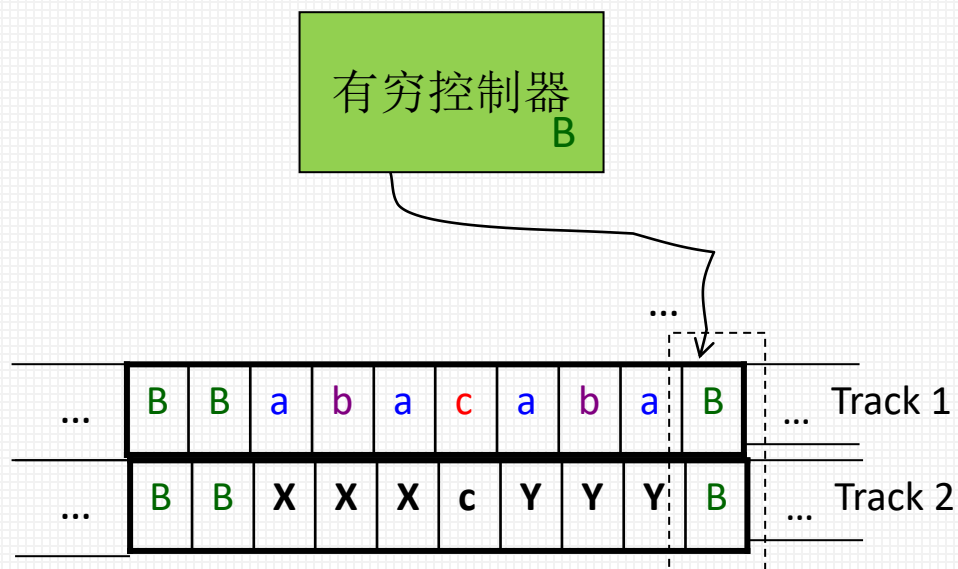


Track 2主要起到做标记的作用。

注：该图灵机在识别该符号串时不需要对其进行修改。

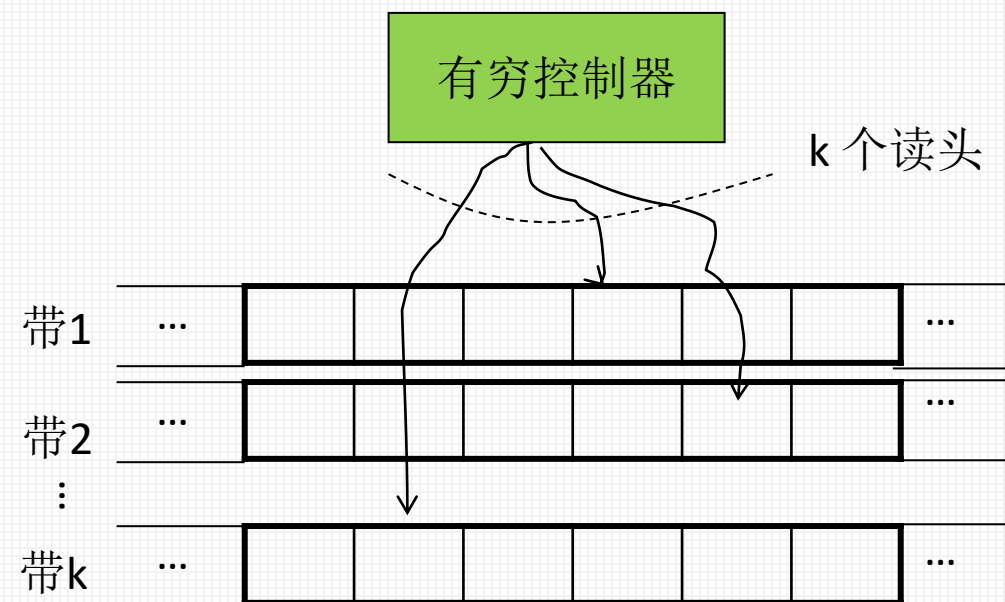
- 例：利用状态中存储与多道设计TM识别

$$L = \{wcw | w \in \{a, b\}^*\}$$

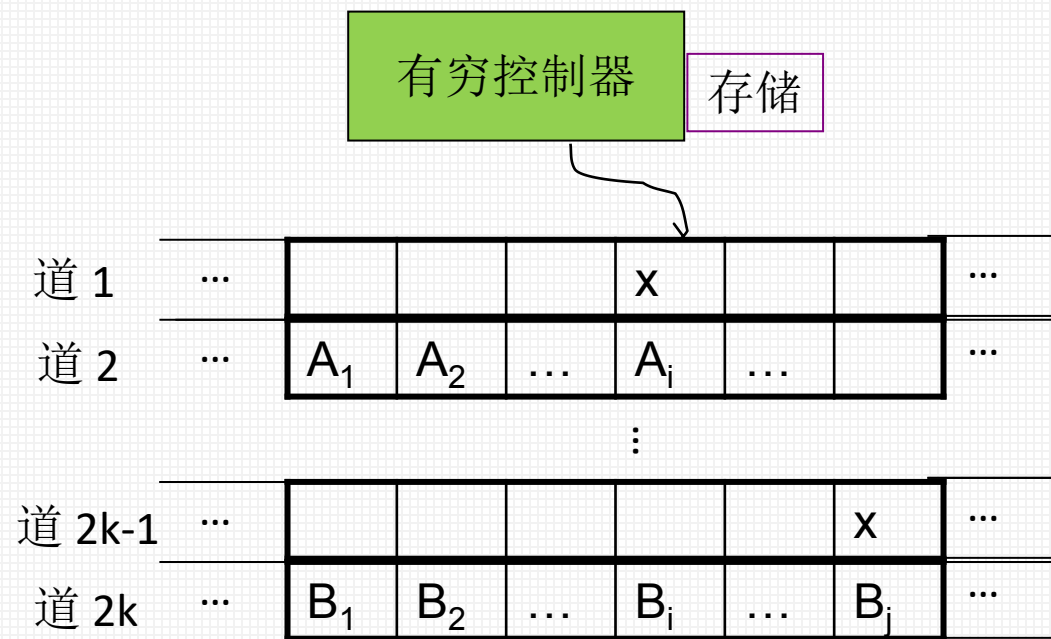


Track 2主要起到做标记的作用。

- 多带(multi-tape)图灵机：有穷控制器、 k 个带头和 k 条带组成。
- 初始时：(1) 输入符号串在第一条带上，其他带上都是空格符。(2) 第一条带的带头指向第一个输入符号。(3) 其他带的带头可以在任意位置
- 每个动作，根据状态和每个带头所指向的符号：
 - 改变控制器中的状态；
 - 修改每个带头单元格中的符号；
 - 每个带头独立地向左或右移动一个单元格，或保持不动(stationary)。



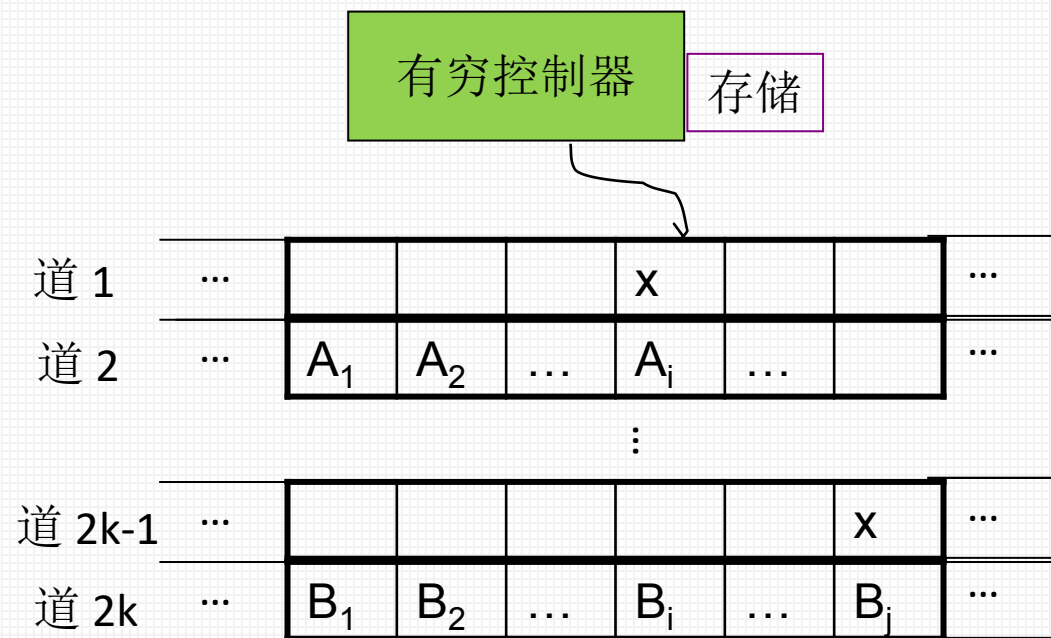
- 定理: 多带图灵机接受的语言 L , 可被单带图灵机接受。



2k道单带图灵机模拟k带图灵机

- 用具有 $2k$ 条道的单带图灵机 M' 模拟 k 带图灵机 M . k 条道对应于 k 条带的内容, 另外 k 条道用于存 k 条带的当前带头位置, 每条道只存 1 个带头标记符号 (head marker)。

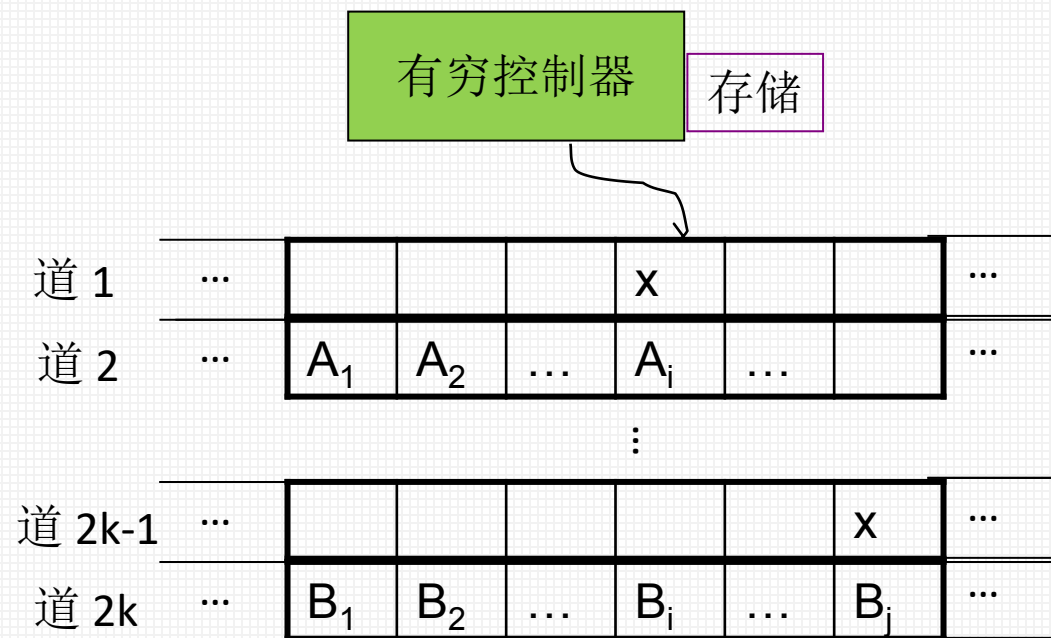
- 定理: 多带图灵机接受的语言 L , 可被单带图灵机接受。



2k道单带图灵机模拟k带图灵机

- 用具有 $2k$ 条道的单带图灵机 M' 模拟 k 带图灵机 M . k 条道对应于 k 条带的内容, 另外 k 条道用于存 k 条带的当前带头位置, 每条道只存 1 个带头标记符号 (head marker)。
- 为模拟 M 的一个动作, M' 需要从左至右, 再从右至左, 各扫描一次;
- 第一次扫描收集存储各带头处符号, 第二次更新每个带头符号和带头位置。

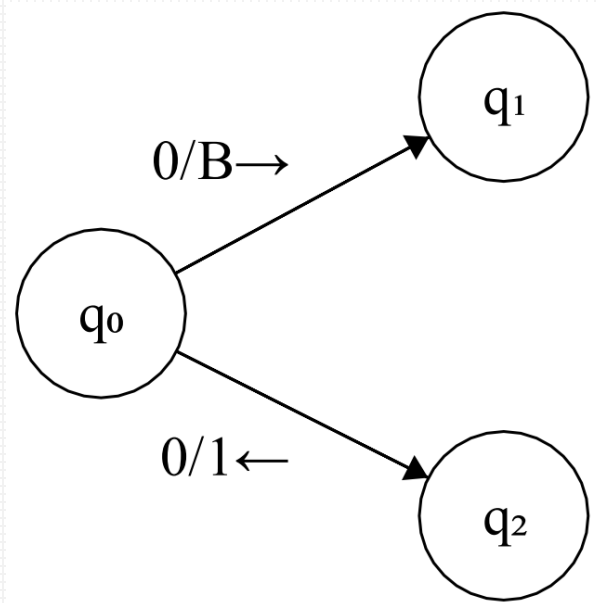
- 定理: 多带图灵机接受的语言 L , 可被单带图灵机接受。



2k道单带图灵机模拟k带图灵机

- 用具有 $2k$ 条道的单带图灵机 M' 模拟 k 带图灵机 M . k 条道对应于 k 条带的内容, 另外 k 条道用于存 k 条带的当前带头位置, 每条道只存 1 个带头标记符号 (head marker)。
- 为模拟 M 的一个动作, M' 需要从左至右, 再从右至左, 各扫描一次;
- 第一次扫描收集存储各带头处符号, 第二次更新每个带头符号和带头位置。
- M' 有穷控制器中状态的转移和 M 一样
- 当被模拟的 M' 进入接受状态, M 也进入接受状态。

- 图灵机在每组状态 q 和带符号 X 的转移 $\delta(q, X)$, 可以有有限个选择:
$$\delta(q, X) = \{(q_1, Y_1, D_1), (q_2, Y_2, D_2), \dots, (q_k, Y_k, D_k)\}$$
- 例: $\delta(q_0, 0) = \{(q_1, B, R), (q_2, 1, L)\}$



(1) NTM接受语言的方式, 与NFA和PDA是类似的
(2) 只要存在从初始ID到某接受状态ID的转移序列, 其他转移序列可以忽略

- **定理：** 如果 L 被非确定图灵机 N 接受， 那么 L 被某个确定的图灵机 M 接受。
- **主要思路：**
 - 让等价的基本TM M 具有3条带。
 - 第1条带用来存放输入。
 - 第2条带上系统地生成 M 的各种可能的移动序列
 - M 在第3条带上复制第一条带上的输入， 然后按照第2条带上给出的移动序列处理输入串， 如果成功， 则接受之， 如果不成功， 则在第2条带上生成下一个可能的移动序列， 开始新一轮的“试处理”。

即让 M 去执行 N 所有可能的分支， 只要在某一条分支进入接受状态停机， 就能接受同样的语言 L 。

- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- 图灵机的表示能力：语言、停机
- 扩展图灵机
- **受限图灵机**

- 半无穷带图灵机：图灵机输入输出带只有一侧是无穷的。
- 定理：半无穷带图灵机，与图灵机等价。

- 证明方法：让一侧无穷的图灵机带使用多道技术，模拟双侧无穷的图灵机带。

双侧
无穷带

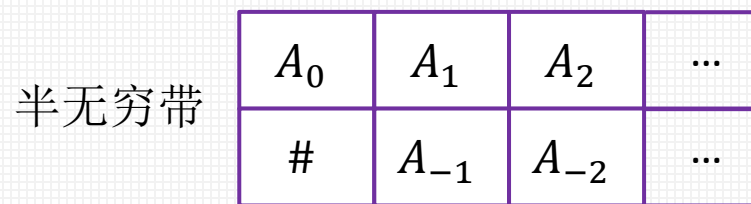
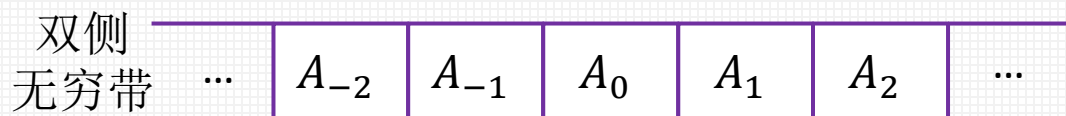
...	A_{-2}	A_{-1}	A_0	A_1	A_2	...
-----	----------	----------	-------	-------	-------	-----

半无穷带

A_0	A_1	A_2	...
#	A_{-1}	A_{-2}	...

当双侧无穷图灵机向右移动时，半无穷带图灵机也正常向右移动。

当双侧无穷图灵机向左移动时，半无穷带图灵机正常也向左移动，而需要越过左边边界位置的时候，通过特定的符号，让半无穷带图灵机进行跳转，向右移动，且使用另一条道即可。



- 令 $M_2 = (Q_2, \Sigma_2, \Gamma_2, \delta_2, q_2, B, F_2)$ 为基本的图灵机。
- 构造半无穷带的两道图灵机 $M_1 = (Q_1, \Sigma_1, \Gamma_1, \delta_1, q_1, B', F_1)$, 则
 - Q_1 包含 (q, U) 和 (q, D) , 对于任意的 $q \in Q_2$
 - U 表示对上面的 Track 进行操作, D 表示对下面的 Track 进行操作
 - 另外 Q_1 包含自己的初始状态 q_1
 - Γ_1 包含所有的二元组 $[X, Y]$, $X \in \Gamma_2, Y \in \Gamma_2 \cup \{\#\}$
 - Σ_1 包含所有的 $[a, B]$, 这里 $a \in \Sigma_2$
 - F_1 为 $\{[q, U], [q, D] | q \in F_2\}$
 - B' 为二元组 $[B, B]$

双侧
无穷带

...	A_{-2}	A_{-1}	A_0	A_1	A_2	...
-----	----------	----------	-------	-------	-------	-----

半无穷带

A_0	A_1	A_2	...
#	A_{-1}	A_{-2}	...

- 对于状态转移函数 δ_1 的定义为：
 - 若 $\delta_2(q_2, a) = (q, X, R)$ ，则令 $\delta_1(q_1, [a, B]) = ([q, U], [X, \#], R)$ 。对应 M_2 的第一步为向右移的情况
 - 若 $\delta_2(q_2, a) = (q, X, L)$ ，则令 $\delta_1(q_1, [a, B]) = ([q, D], [X, \#], R)$ 。对应 M_2 的第一步为向左移的情况

双侧
无穷带

...	A_{-2}	A_{-1}	A_0	A_1	A_2	...
-----	----------	----------	-------	-------	-------	-----

半无穷带

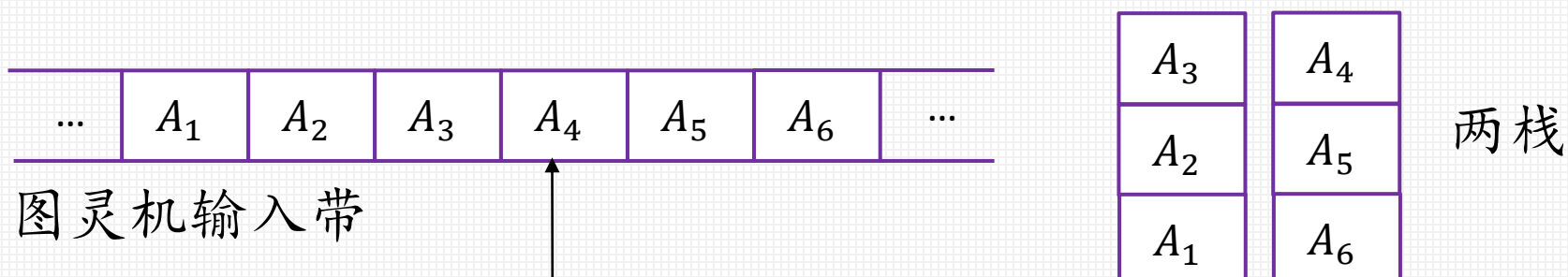
A_0	A_1	A_2	...
#	A_{-1}	A_{-2}	...

- 对于状态转移函数 δ_1 的定义为：
 - 对于任意的 $[X, Y] \in \Gamma_1$, Y 不为#, $A = L$ 或者 $A = R$
 - 若 $\delta_2(q, X) = (p, Z, A)$, 则令 $\delta_1([q, U], [X, Y]) = ([p, U], [Z, Y], A)$ 。
对应在第1条道模拟 M_2 的情况。
 - 若 $\delta_2(q, Y) = (p, Z, \bar{A})$, 则令 $\delta_1([q, D], [X, Y]) = ([p, D], [X, Z], A)$ 。
对应在第2条道模拟 M_2 的情况。
 - 若 $\delta_2(q, X) = (p, Y, A)$, 则令 $\delta_1([q, U], [X, \#]) = \delta_1([q, D], [X, \#]) = ([p, C], [Y, \#], R)$ 。这里如果 $A = R, C = U$;如果 $A = L, C = D$ 。能让 M_1 在左移到最左边的单元格的时候, 进行道的切换。

- 多栈机器：基于下推自动机的扩展， k 栈机器是具有 k 个栈的确定型下推自动机。
- 定理：如果图灵机接受 L ，那么双栈机接受 L 。

- **证明要点：**

- 用两个堆栈模拟图灵机带，一个堆栈保存带头左边内容，一个堆栈保存带头右边内容，但都不包括无穷的空格符号。
- 带头的移动用两个栈分别弹栈和压栈模拟。若带头修改字符 A 为 B ，用一个栈弹出的字符 A 而另一个栈压入 B 来模拟。
- 开始时输入在双栈机的输入带，可以先将输入扫描并压入一个栈，然后再依次把每个符号弹出再压入另一个栈，然后进入模拟图灵机的状态。



- 图灵机简介、形式化定义
- 基于图灵机的整数函数运算
- 图灵机的表示能力：语言、停机
- 扩展图灵机
 - 状态具有有穷存储功能的TM、多道TM、多带TM、非确定型TM
- 受限图灵机
 - 半无穷带、多栈机器